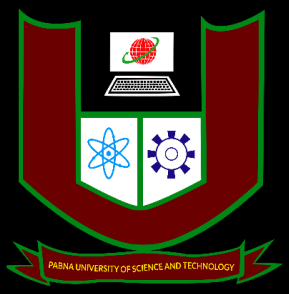
Pabna University of Science and Technology



Department of

Information and Communication Engineering

Lab Report

Course Title: Digital Image and Speech Processing Sessional

Course Code: ICE-3208

Submitted By:	**Submitted To:**
Name: Umme Habiba Antora **Roll: 230637** Session:2022-2023 Semester:3rd year 2nd Semester Dept. of Information and Communication Engineering Pabna University of Science and Technology	**Dr. Md. Imran Hossain** Associate Professor Department of Information and Communication Engineering Pabna University of Science and Technology

Submission Date: 01-11-2026

INDEX

Exp. No.	Name of the Experiment
1	Display following image operation in MATLAB/Python - i) Histogram image ii) Low pass filter image iii) High pass image.
2	Write a MATLAB/Python program to read ‘rice.tif’ image, count number of rice and display area (also specific range), major axis length, and perimeter.
3	Write a MATLAB/Python program to read an image and perform convolution with 3X3 mask.
4	Write a MATLAB/Python program to read an image and perform Laplacian filter mask.
5	Write a MATLAB/Python program to Character Segment of an image.
6	For the given image perform edge detection using different operators and compare the results.
7	Write a MATLAB/Python program to read coins.png, leveling all coins and display area of all coins.
8	Display following image operation in MATLAB/Python - i) Threshold image ii) Power enhance contract image iii) High pass image.
9	Perform image enhancement, smoothing and sharpening, in spatial domain using different spatial filters and compare the performances.
10	Write a MATLAB/Python program to separation of voiced/un-voiced/silence regions from a speech signal.
11	Write a MATLAB/Python program and plot multilevel speech resolution.
12	Write a MATLAB/Python program to recognize speech signal.
13	Write a MATLAB/Python program for text-to-speech conversion and record speech signal.

Experiment No: 01

Name of the Experiment: To display image operations in Python – i) Histogram Image ii) Low Pass Filter Image iii) High Pass Filter Image.

Objectives:

1. To learn how to read and display an image in Python using OpenCV.
2. To generate and understand the histogram of a grayscale image.
3. To apply a low pass filter to blur/smooth an image.
4. To apply a high pass filter to highlight the edges of an image.
5. To compare the original image with the filtered images and understand the effect of each filter.

Theory:

1. Digital Image:
A digital image can be represented as a two-dimensional function f(x, y), where x and y represent the position or coordinates of a point in the image, and f(x, y) represents the intensity or gray level at that point.
In a grayscale image, the intensity value usually ranges from 0 to 255. Here, **0** represents black, 255 represents white and values between 0 and 255 represent different shades of gray. An 8-bit image has 256 possible gray levels, from 0 to 255.

2. Histogram of an Image:
An image histogram is a graph that shows how the pixel intensity values are distributed in an image.

- The x-axis represents the intensity levels, from 0 to 255 for an 8-bit image.
- The y-axis represents the frequency, or the number of pixels having each intensity value.

Mathematically, the histogram of a digital image is defined as:

$h(\mathbf{r}_k) = \mathbf{n}_k$
where,
$\mathbf{r}_k$ = the k-th gray level
$\mathbf{n}_k$ = the number of pixels having the gray level r_k.
A histogram helps us understand the brightness and contrast of an image. A narrow histogram generally indicates low contrast. A wide and spread-out histogram generally indicates high contrast.

3. Low Pass Filter (LPF):
A Low Pass Filter (LPF) allows the low-frequency components of an image to pass and reduces the high-frequency components.

In an image, low-frequency components represent smooth areas that change slowly, such as backgrounds and high-frequency components represent sharp changes, such as edges, noise, and fine details. Therefore, low pass filtering is mainly used for smoothing or blurring an image and reducing noise. Some common low pass filters are Average filter, Gaussian filter, and Median filter. A low pass filter commonly uses a small matrix called a kernel. For example, a 3×3 averaging kernel is:

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

This kernel replaces each pixel with the average value of its neighboring pixels. As a result, the image becomes smoother and slightly blurred.

4. High Pass Filter (HPF):
A High Pass Filter (HPF) allows the high-frequency components of an image to pass and reduces the low-frequency components. Edges, fine details, and sudden changes in intensity are mainly high-frequency information. Therefore, high pass filters are mainly used for edge detection, image sharpening and enhancing fine details.

A commonly used high pass filter kernel is:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

This kernel compares a pixel with its surrounding 8 neighboring pixels. The center pixel is multiplied by 8, while the surrounding pixels are given −1.

The sum of all kernel values is zero: $8-8 = 0$ .Therefore, smooth or flat areas are largely removed, while edges and sudden intensity changes are emphasized.

5. Relationship between LPF and HPF:
Low pass and high pass filters work in opposite ways.The high-frequency component can also be obtained by subtracting the low-frequency component from the original image:

High Pass Image = Original Image − Low Pass Image

In other words, when the smooth/low-frequency part is removed from the original image, the remaining part mainly contains edges and fine details.

Source code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read Image
img = cv2.imread('cameraman.jpg')

# Check if image loaded properly
if img is None:
    print("Error: Image not found. Check the file path/name.")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# i) Histogram
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])

# ii) Low Pass Filter
low_pass = cv2.GaussianBlur(gray, (5, 5), 0)

# iii) High Pass Filter
kernel = np.array([
    [-1, -1, -1],
    [-1,  8, -1],
    [-1, -1, -1]
])

high_pass = cv2.filter2D(gray, -1, kernel)

# Display Results
plt.figure(figsize=(12, 8))
```

```
# Original Image
plt.subplot(2, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title('Original Grayscale Image')
plt.axis('off')

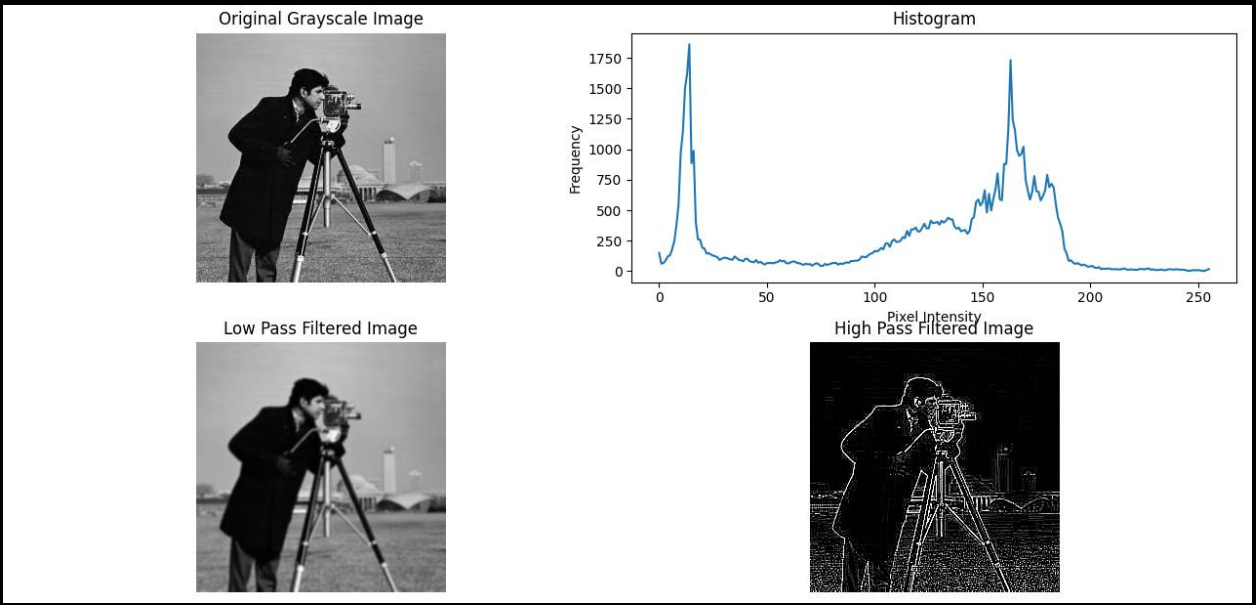
# Histogram
plt.subplot(2, 2, 2)
plt.plot(hist)
plt.title('Histogram')
plt.xlabel('Pixel Intensity')
plt.ylabel('Frequency')

# Low Pass Filter
plt.subplot(2, 2, 3)
plt.imshow(low_pass, cmap='gray')
plt.title('Low Pass Filtered Image')
plt.axis('off')

# High Pass Filter
plt.subplot(2, 2, 4)
plt.imshow(high_pass, cmap='gray')
plt.title('High Pass Filtered Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Output:



Result:

The histogram, low pass filtered image, and high pass filtered image were successfully obtained using Python. The histogram shows the distribution of pixel intensity values, while the filtered images show the effects of low pass and high pass filtering.

Discussion:

The histogram helps to understand the brightness and contrast of the image. The low pass filter reduces high-frequency components and makes the image smoother. On the other hand, the high pass filter emphasizes edges, fine details, and sudden changes in intensity. Thus, the two filters produce different effects on the same image.

Conclusion:

The experiment successfully demonstrated histogram analysis, low pass filtering, and high pass filtering of a digital image using Python. The results show that LPF is useful for smoothing, while HPF is useful for edge enhancement.

Experiment No: 02

Name of the Experiment: To Write a Python Program to Read ‘rice.tif’ Image, Count the Number of Rice Grains, and Display Area (Also Specific Range), Major Axis Length, and Perimeter

Objectives:

1. To understand the concept of object counting and region-based measurement in image processing.
2. To learn how to separate rice grains from the background using thresholding.
3. To count the number of rice grains using connected component labeling.
4. To measure the area, major axis length, and perimeter of each rice grain.
5. To display only those rice grains whose area lies within a specific range.

Theory:

1. Digital Image:
A digital image is represented as a two-dimensional function f(x, y), where x and y are the position (coordinates) of a point in the image, and the value of f at that point is called the intensity or gray level. In a grayscale image, this intensity value usually ranges from 0 to 255, where 0 means black, 255 means white, and the values in between represent different shades of gray.

2. Thresholding (Otsu’s Method):
Thresholding converts a grayscale image into a binary image, where objects become white (255) and the background becomes black (0). Otsu’s method is an automatic thresholding technique that selects the best threshold value by minimizing the variation of pixel intensities within the object group and the background group. Because of this, there is no need to select the threshold value manually. In this experiment, the rice grains are the bright objects, so they become white in the binary image.

3. Morphological Opening:
Opening is a morphological operation that consists of erosion followed by dilation. Erosion shrinks the white regions and removes very small white spots, and dilation then expands the remaining regions back to their original size. As a result, small noise is removed from the binary image while the main objects (rice grains) keep their shape and size. In this experiment, a 3×3 kernel is used for this operation.

4. Connected Component Labeling:
Connected component labeling is the process of assigning a unique label

(number) to each group of connected white pixels in a binary image. Each connected group represents one object. In this experiment, each rice grain gets a unique label, and the total number of labels gives the number of rice grains present in the image.

5. Region Properties:
After labeling, different measurements can be calculated for each object using region properties:

- **Area:** The total number of pixels that belong to the object. It shows the size of each rice grain.
- **Major Axis Length:** The length of the major axis of the ellipse that has the same normalized second central moments as the object. For a rice grain, it gives approximately the length of the grain.
- **Perimeter:** The total length of the boundary (outline) of the object. It shows how long the edge of the rice grain is.

6. Removing Small Objects:
Very small objects (for example, area less than 50 pixels) are considered noise and are removed from the measurement list. This makes sure that only real rice grains are counted.

7. Area Range Filtering:
To analyze grains of a particular size, a minimum and a maximum area value is set (here 100 to 500 pixels), and only those objects whose area lies within this range are selected and displayed. This helps in separating normal-size grains from very small objects (broken pieces or noise) and very large objects (touching or merged grains).

8. Working Principle:

- The rice image is read in grayscale and displayed.
- The image is converted into a binary image using Otsu's thresholding.
- Small noise is removed using morphological opening.
- Connected component labeling is applied to find each rice grain.
- Very small objects are removed and the remaining objects are counted.
- Area, major axis length, and perimeter of each grain are calculated and printed.
- Grains whose area is within the specified range are selected and displayed in a separate image.

9. Applications:

- Quality inspection of grains in the food industry

- Counting and sorting objects such as seeds, cells, and coins
- Measuring the size and shape of objects in medical and industrial images
- Automated agricultural analysis

Source code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.measure import label, regionprops

# Read rice image
image = cv2.imread("rice.tif", cv2.IMREAD_GRAYSCALE)

# Display original image
plt.figure(figsize=(6, 6))
plt.imshow(image, cmap="gray")
plt.title("Original Rice Image")
plt.axis("off")
plt.show()

# Convert image to binary
_, binary = cv2.threshold(
    image, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# Remove small noise
kernel = np.ones((3, 3), np.uint8)
binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)

# Label connected rice grains
label_image = label(binary > 0)

# Measure properties
properties = regionprops(label_image)

# Remove very small objects
properties = [p for p in properties if p.area > 50]

# Number of rice grains
number_of_rice = len(properties)
```

```
print("Number of rice grains:", number_of_rice)
print()

# Display measurements
print("Rice No.\tArea\t\tMajor Axis Length\tPerimeter")
print("-" * 65)

for i, prop in enumerate(properties, start=1):
    print(
        f"{i}\t\t"
        f"{prop.area:.2f}\t\t"
        f"{prop.major_axis_length:.2f}\t\t\t"
        f"{prop.perimeter:.2f}"
    )

# ---------------------------------------
# Specific Area Range
# ---------------------------------------

min_area = 100
max_area = 500

selected_rice = [
    p for p in properties
    if min_area <= p.area <= max_area
]

print()
print(f"Rice with area between {min_area} and {max_area}: "
      f"{len(selected_rice)}")

# Display selected rice grains
selected_image = np.zeros_like(label_image)

for p in selected_rice:
    selected_image[label_image == p.label] = 1

plt.figure(figsize=(6, 6))
plt.imshow(selected_image, cmap="gray")
plt.title(
    f"Rice with Area {min_area}-{max_area} pixels"
)
```

```
plt.axis("off")
plt.show()
```

Output:

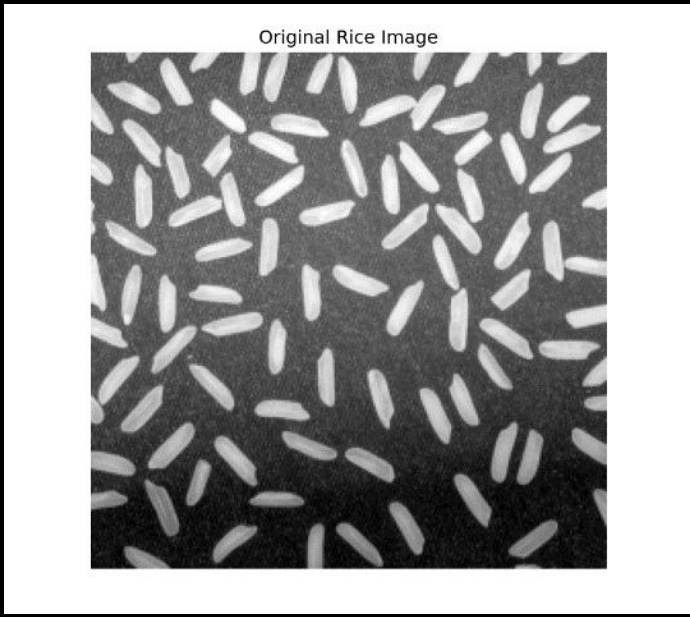
Number of rice grains: 83

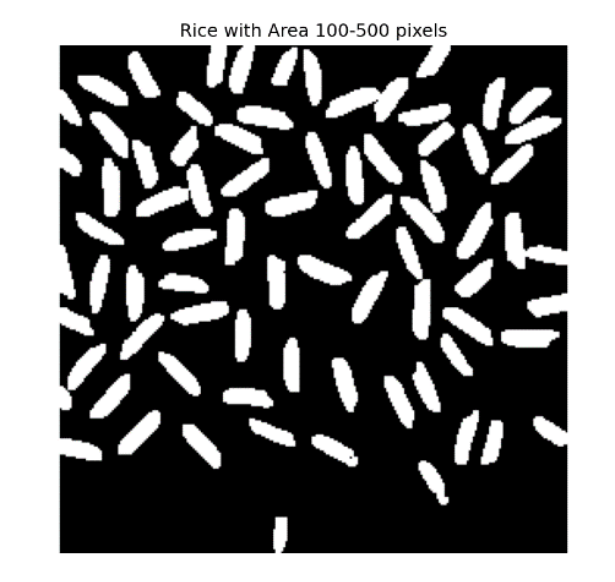
Rice No.	Area	Major Axis Length	Perimeter
1	85.00	14.37	34.73
2	163.00	23.27	53.31
3	195.00	26.91	61.21
4	89.00	14.95	36.73
5	154.00	21.79	52.04
6	65.00	11.61	29.66
7	66.00	14.38	31.28
8	144.00	22.10	51.21
9	188.00	29.03	62.97
10	197.00	26.97	61.11
11	213.00	28.52	64.28
12	425.00	45.79	122.81
13	195.00	28.25	62.38
14	193.00	26.74	58.18
15	113.00	19.55	45.80
16	162.00	23.27	52.53
17	188.00	28.00	61.80
18	232.00	29.90	67.21
19	210.00	28.92	63.60
20	208.00	30.50	67.70
21	152.00	23.25	50.77
22	205.00	27.51	63.46
23	188.00	27.25	60.28
24	159.00	21.37	51.70
25	199.00	31.08	66.87
26	224.00	30.85	67.01
27	186.00	26.39	60.63
28	193.00	28.26	61.01
29	211.00	30.61	65.56
30	104.00	15.68	38.97
31	222.00	30.95	67.31
32	206.00	30.52	65.60
33	442.00	40.38	131.05
34	419.00	43.81	130.12

35	88.00	13.20	35.31
36	221.00	28.94	65.21
37	220.00	31.57	68.43
38	248.00	31.96	72.18
39	225.00	30.18	66.14
40	183.00	28.52	62.53
41	370.00	40.30	115.11
42	404.00	65.26	131.60
43	194.00	28.71	63.21
44	139.00	28.42	60.73
45	194.00	30.35	66.38
46	209.00	29.04	63.56
47	229.00	29.48	67.11
48	192.00	24.46	56.77
49	205.00	28.32	63.56
50	221.00	30.90	68.77
51	173.00	26.17	58.38
52	167.00	22.22	53.56
53	222.00	30.65	68.63
54	139.00	20.80	49.80
55	218.00	31.16	68.18
56	183.00	27.60	59.31
57	203.00	31.45	67.84
58	223.00	29.89	66.73
59	150.00	26.13	55.94
60	191.00	28.66	61.31
61	253.00	41.73	90.67
62	92.00	15.18	37.56
63	193.00	29.96	63.25
64	210.00	29.10	65.46
65	183.00	27.13	62.28
66	208.00	29.13	63.60
67	201.00	29.51	65.11
68	62.00	10.97	28.24
69	186.00	26.86	59.56
70	217.00	30.11	63.84
71	191.00	29.47	65.80
72	145.00	21.28	48.87
73	157.00	27.15	59.46
74	158.00	24.85	55.56
75	202.00	28.51	61.01
76	179.00	27.34	65.94
77	169.00	23.85	54.73

78	91.00	18.56	41.56
79	150.00	25.67	57.70
80	58.00	15.63	32.24
81	96.00	20.93	45.80
82	96.00	18.24	41.80
83	108.00	19.46	43.07

Rice with area between 100 and 500: 72





Result:

The ‘rice.tif’ image was successfully processed using the Python program. The following results were obtained:

- Total number of rice grains detected: 83
- Area, major axis length, and perimeter of each rice grain were calculated and displayed in a table.
- Number of rice grains with area between 100 and 500 pixels: 72

Discussion:

In this experiment, the rice.tif image was converted to grayscale and then to a binary image using Otsu’s thresholding. Morphological opening with a 3×3 kernel removed small noise. Connected component labeling counted the rice grains, excluding objects smaller than 50 pixels. The area, major axis length, and perimeter of each grain were calculated using region properties. Grains with

areas between 100 and 500 pixels were selected and displayed. Uneven lighting may cause grains to be missed, while touching grains may be counted as one. Background removal and watershed segmentation can reduce these problems.

Conclusion:

The experiment successfully counted rice grains and measured their area, major axis length, and perimeter. It demonstrated how thresholding, morphological opening, connected component labeling, and region properties can be used for effective object counting, measurement, and size-based filtering.

Experiment No.: 03

Name of the Experiment: To read an image and perform convolution using a 3×3 mask in Python.

Objectives:

1. To understand the basic concept of image convolution.
2. To read and display a digital image using Python.
3. To perform convolution on an image using a 3×3 mask.
4. To observe the effect of the convolution mask on an image.
5. To understand how convolution can be used for edge detection and image processing.

Theory:

1. Digital Image:
A digital image can be represented as a two-dimensional function f(x, y)**,** where x and y represent the position or coordinates of a pixel, and f(x, y) represents the intensity value at that position. In a grayscale image, the intensity value usually ranges from 0 to 255, where 0 represents black and 255 represents white.

2. Convolution:
Convolution is an important operation in digital image processing. It is used to modify or extract useful information from an image by applying a small matrix called a mask, kernel, or filter.
In convolution, the mask is moved over the image pixel by pixel. At each position, the values of the image pixels in the selected region are multiplied by the corresponding values of the mask. The results are then added together to obtain a new pixel value.

For a 3×3 mask, the operation can be represented as:

$$g(x,y) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} f(x+i, y+j) h(i,j)$$

where f(x, y) is the input image, h(i, j) is the 3×3 mask, and g(x, y) is the output image.

3. 3×3 Mask:
A 3×3 mask contains nine values arranged in three rows and three columns. The mask is applied to a small 3×3 region of the image at a time. Different masks produce different effects such as smoothing, sharpening, and edge detection.

In this experiment, the following 3×3 mask is used:

-1 -1 -1
-1 8 -1
-1 -1 -1

This is a **high-pass or edge detection mask**. The center value is 8 and all surrounding values are −1. The sum of all the mask values is zero: 8 – 8 = 0 Therefore, smooth areas are reduced, while areas with sudden changes in intensity, such as edges and boundaries, are emphasized.

4. Working Principle of the Mask:

The 3×3 mask is moved over the image from one pixel to another. For each position, the 3×3 image region is multiplied element by element with the 3×3 mask, and all the resulting values are added together. This calculated value becomes the corresponding pixel value of the output image.
For example, if a 3×3 region of an image is:

a b c
d e f
g h i

then the convolution with the above mask produces:
(− a − b – c – d + 8e – f – g – h − i)
This process is repeated for the whole image to produce the final convolved image.

Source code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("cameraman.jpg", cv2.IMREAD_GRAYSCALE)

# 3x3 mask
mask = np.array([
    [-1, -1, -1],
    [-1,  8, -1],
    [-1, -1, -1]
])

# Get image dimensions
rows, cols = image.shape

# Create output image
output = np.zeros((rows, cols), dtype=np.float32)

# Perform convolution manually
for i in range(1, rows - 1):
    for j in range(1, cols - 1):

        region = image[i-1:i+2, j-1:j+2]

        output[i, j] = np.sum(region * mask)

# Normalize result for display
output = cv2.normalize(
    output,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# Display
plt.figure(figsize=(10, 4))

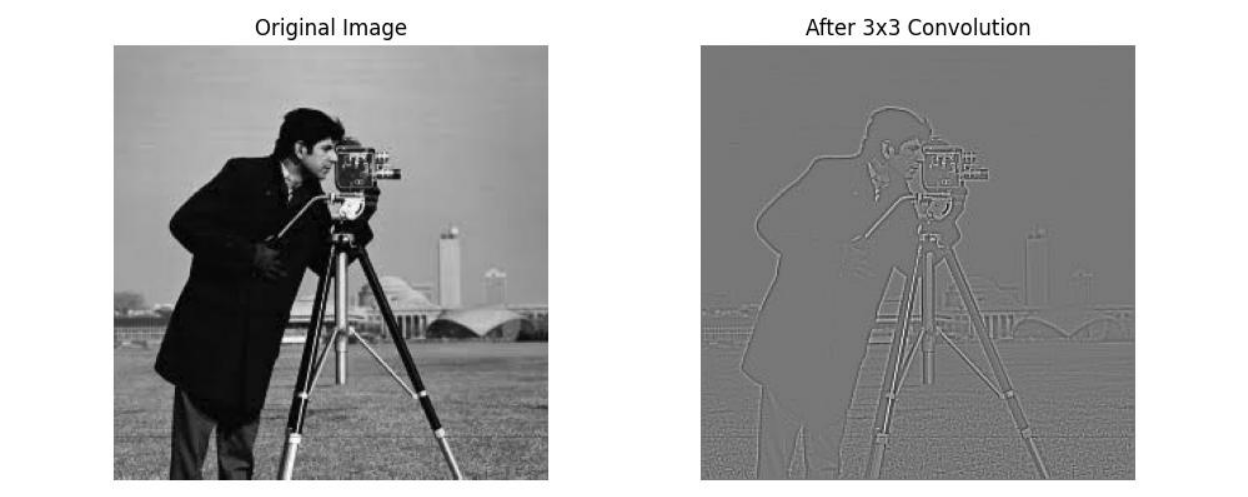
plt.subplot(1, 2, 1)
```

```
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(output, cmap="gray")
plt.title("After 3x3 Convolution")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:



Result:

The convolution of the image using a 3×3 mask was successfully performed. The output image shows the edges and fine details more clearly than the original image.

Discussion:

The 3×3 high-pass mask emphasizes sudden changes in pixel intensity and reduces smooth regions. As a result, the edges and boundaries of objects become more visible in the output image.

Conclusion:

The experiment successfully demonstrated image convolution using a 3×3 mask in Python. The result shows that convolution can be used effectively for detecting edges and enhancing important details in a digital image.

Experiment No.: 04

Experiment Name: To read an image and perform Laplacian filter mask in Python.

Objectives:

1. To read and display a grayscale image using Python.
2. To understand the concept of Laplacian filtering.
3. To apply a 3×3 Laplacian filter mask to an image.
4. To detect edges and sudden intensity changes in an image.
5. To observe the effect of Laplacian filtering on the original image.

Theory:

1. Digital Image :

A digital image is represented as a two-dimensional function f(x,y) where x and y represent the spatial coordinates of a pixel. For a grayscale image, the intensity value usually ranges from 0 to 255. Here, 0 represents black and 255 represents white.

2. Laplacian Filter :

The Laplacian filter is an image processing technique used mainly for detecting edges and fine details in an image. It is based on the second-order derivative of the image intensity. It detects rapid changes in intensity between neighboring pixels.
The Laplacian operator can be represented as:

$$\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$$

It responds strongly at the boundaries of objects where the intensity changes suddenly.

3. Laplacian Filter Mask :

A commonly used 3×3 Laplacian mask is:

$$\begin{matrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{matrix}$$

Another common version (8-connectivity) is:

-1 -1 -1
-1 8 -1
-1 -1 -1

The center value is positive and the neighboring values are negative. The sum of all mask values is zero. Therefore, smooth regions with nearly constant intensity produce a small response, while regions containing edges produce a stronger response.

4. Working Principle:

- The Laplacian mask is placed over a pixel and its 8 neighboring pixels.
- Each mask value is multiplied with the pixel value beneath it.
- All the multiplication results are added together.
- The final sum becomes the new pixel value in the output image.
- This process is repeated for every pixel by sliding the mask across the entire image.
- The resulting output image highlights the edges, since the Laplacian filter is sensitive to sudden intensity changes.

5. Application of Laplacian Filter:
The Laplacian filter is mainly used for edge detection and image sharpening (by adding the Laplacian result back to the original image). Since the Laplacian filter is sensitive to noise (because it uses second-order derivatives), the image is sometimes smoothed with a low pass filter (like Gaussian blur) before applying the Laplacian filter, to reduce unnecessary noise in the output.

Source code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("cameraman.jpg", cv2.IMREAD_GRAYSCALE)

# Define 3x3 Laplacian filter mask
mask = np.array([
    [0, -1, 0],
    [-1, 4, -1],
    [0, -1, 0]
])

# Apply Laplacian filter
laplacian = cv2.filter2D(image, cv2.CV_64F, mask)

# Convert result to displayable image
laplacian = np.absolute(laplacian)
laplacian = np.uint8(np.clip(laplacian, 0, 255))

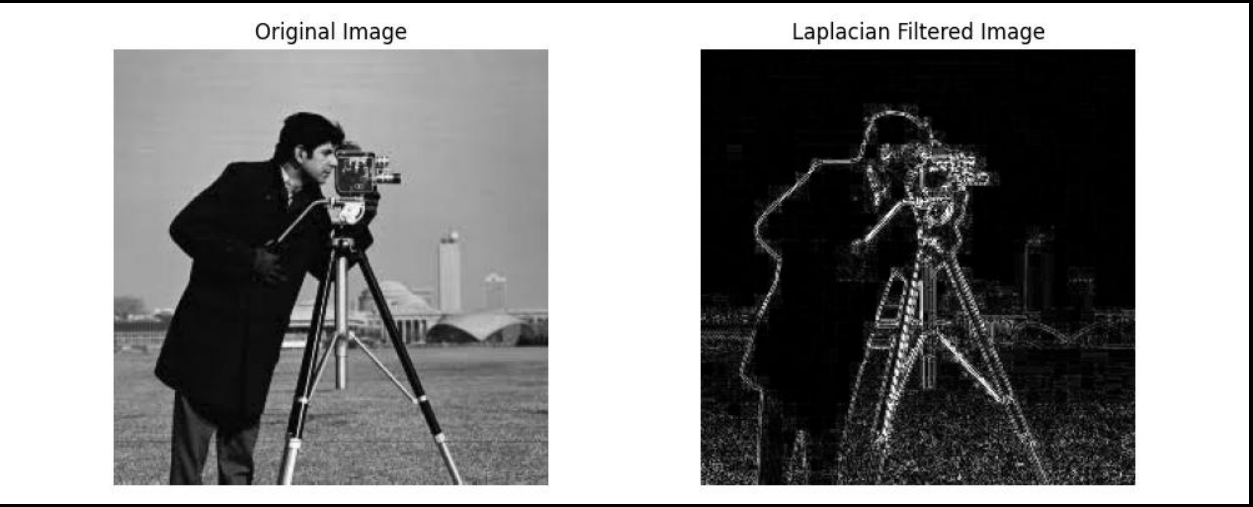
# Display results
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:



Result:

The Laplacian filter was successfully applied to the grayscale image using Python. The filtered image clearly highlights the edges and fine details of the original image.

Discussion:

The Laplacian filter detects sudden changes in image intensity and emphasizes the boundaries of objects. Smooth regions produce a lower response, while edges produce a stronger response. Therefore, the Laplacian filter is useful for edge detection and detail enhancement.

Conclusion:

The Laplacian filtering operation was successfully performed using a 3×3 mask in Python. The experiment demonstrated that the Laplacian filter can effectively detect edges and fine details in a grayscale image.

Experiment No.: 05

Experiment Name: To perform character segmentation of an image using Python.

Objectives:

1. To read and display an image using Python.
2. To convert the image into a binary image.
3. To identify individual characters from the binary image.
4. To remove small unwanted noise from the image.
5. To segment and display individual characters separately.

Theory:

Character Segmentation:
Character segmentation is an important step in image processing and Optical Character Recognition (OCR). It is the process of separating individual characters from a text image so that each character can be processed or recognized separately.

Binary Image :
A binary image contains only two intensity values, usually 0 and 255. In character segmentation, the image is converted into binary form so that the characters can be separated from the background. In this experiment, the image is inverted so that the characters appear as white regions on a black background.

Connected Components :
Connected component analysis is used to identify separate connected regions in a binary image. Each connected region is considered as a component. In a text image, individual characters can often be represented as separate connected components.
The 8-connectivity method considers a pixel connected to its horizontal, vertical, and diagonal neighboring pixels. This helps to identify complete characters as connected regions.

Character Extraction :
After finding the connected components, the position and size of each component are obtained using its bounding box. Very small components are removed as noise. The remaining components are cropped from the binary image and treated as individual characters.

The segmented characters are then arranged from left to right according to their position in the original image and displayed separately.

Importance of Character Segmentation :
Character segmentation is useful in OCR systems, document analysis, number plate recognition, handwriting recognition, and text processing. Proper segmentation makes it easier for a recognition system to identify individual characters correctly.

Source code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ---------------------------------
# 1. Read the image
# ---------------------------------
image = cv2.imread("image6.jpg", cv2.IMREAD_GRAYSCALE)

# Display original image
plt.figure(figsize=(10, 4))
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.show()

# ---------------------------------
# 2. Convert image to binary
# ---------------------------------
_, binary = cv2.threshold(
    image, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Display binary image
plt.figure(figsize=(10, 4))
plt.imshow(binary, cmap="gray")
plt.title("Binary Image")
plt.axis("off")
plt.show()

# ---------------------------------
# 3. Find connected components
# ---------------------------------
```

```
num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
    binary,
    connectivity=8
)

# List to store character images
characters = []

# ----------------------------------
# 4. Extract individual characters
# ----------------------------------
for i in range(1, num_labels):

    x = stats[i, cv2.CC_STAT_LEFT]
    y = stats[i, cv2.CC_STAT_TOP]
    w = stats[i, cv2.CC_STAT_WIDTH]
    h = stats[i, cv2.CC_STAT_HEIGHT]
    area = stats[i, cv2.CC_STAT_AREA]

    # Remove very small noise
    if area > 20:

        character = binary[y:y+h, x:x+w]

        characters.append((x, y, w, h, character))

# Sort characters from left to right
characters.sort(key=lambda c: c[0])

# ----------------------------------
# 5. Display segmented characters
# ----------------------------------

print("Number of segmented characters:", len(characters))

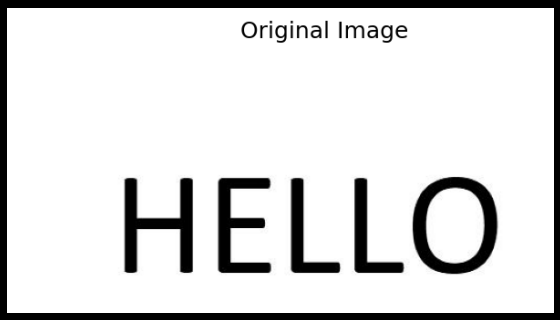
plt.figure(figsize=(12, 4))

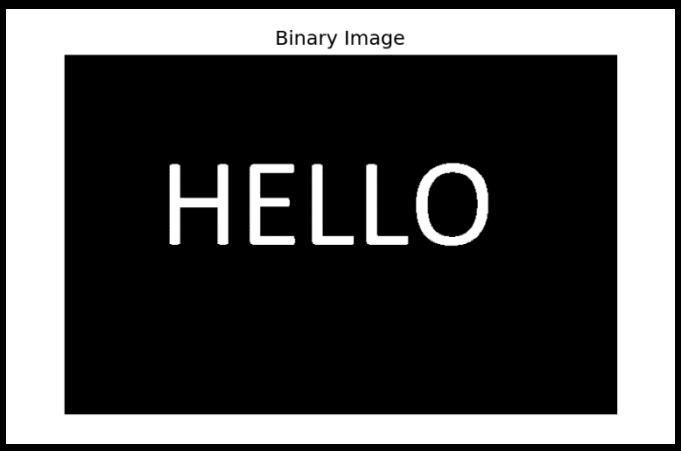
for i, (x, y, w, h, character) in enumerate(characters):

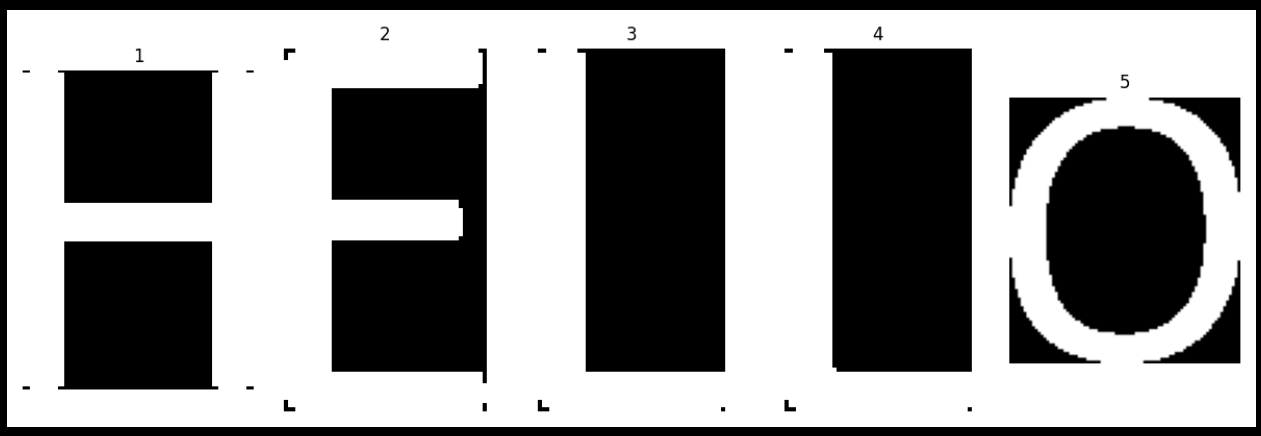
    plt.subplot(1, len(characters), i + 1)
    plt.imshow(character, cmap="gray")
    plt.title(str(i + 1))
    plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Output:







Result:

Character segmentation was successfully performed using Python. The input image was converted into a binary image, individual connected components were identified, small noise components were removed, and the characters were extracted and displayed separately.

Discussion:

The experiment shows that connected component analysis can be used to separate individual characters from a binary text image. Binary conversion makes the characters easier to identify, while connected components help locate each character. Removing small components reduces unwanted noise and improves the segmentation result.

Conclusion:

Character segmentation was successfully performed using Python. The experiment demonstrated that binary thresholding and connected component analysis can effectively separate individual characters from an image.

Experiment No.: 06

Experiment Name: To perform edge detection using different operators and compare the results.

Objectives:

1. To read and display a grayscale image using Python.
2. To understand the concept of edge detection.
3. To detect edges using Sobel, Prewitt, Roberts, and Canny operators.
4. To compare the edge detection results obtained from different operators.
5. To understand the advantages and limitations of different edge detection methods.

Theory:

Digital Image:

A digital image is represented as a two-dimensional function f(x,y), where x and y are the position (coordinates) of a point in the image, and the value of ff at that point is called the intensity or gray level. In a grayscale image, this intensity value usually ranges from 0 to 255, where 0 means black, 255 means white, and the values in between represent different shades of gray.

Edge Detection:

Edge detection is a fundamental technique in image processing used to identify points in an image where the intensity changes sharply or has discontinuities. These points usually correspond to boundaries of objects, surfaces, or important features within the image. Edge detection helps in reducing the amount of data in an image while preserving the important structural information.

Edges occur due to sudden changes in pixel intensity, and mathematically, they can be detected by calculating the derivative or gradient of the image intensity function. A large gradient value indicates a strong edge, while a small gradient value indicates a smooth or flat region.

Sobel Operator:

The Sobel operator is a first-order derivative-based edge detection method that calculates the gradient of image intensity in the horizontal and vertical directions using two 3×3 kernels.

Horizontal (Gx):

-1 0 1
-2 0 2
-1 0 1

Vertical (Gy):

$$\begin{matrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{matrix}$$

The overall gradient magnitude is calculated as:

$G = \sqrt{(Gx^2 + Gy^2)}$

The Sobel operator gives more weight to the pixels closer to the center, which makes it slightly less sensitive to noise compared to simpler derivative operators.

Prewitt Operator:

The Prewitt operator is similar to the Sobel operator and is also used to detect horizontal and vertical edges using two 3×3 kernels.

Horizontal (Gx):

$$\begin{matrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{matrix}$$

Vertical (Gy):

$$\begin{matrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{matrix}$$

The Prewitt operator gives equal weight to the neighboring pixels. It is simple and useful for detecting edges, but it can be more sensitive to noise than the Sobel operator.

Roberts Operator:

The Roberts operator is a simple first-order derivative-based edge detection method that uses two 2×2 kernels to detect intensity changes in diagonal directions.

The two masks are:

$$\begin{matrix} 1 & 0 \\ 0 & -1 \end{matrix}$$

and

$$\begin{matrix} 0 & 1 \\ -1 & 0 \end{matrix}$$

The results from the two masks are combined to obtain the edge magnitude. The Roberts operator is computationally simple and can detect sharp edges, but because it uses a small 2×2 mask, it is more sensitive to noise.

Canny Edge Detector:

The Canny edge detector is a more advanced, multi-step edge detection algorithm. It generally produces thin, clean, and well-defined edges. The main

steps of the Canny method include noise reduction using a Gaussian filter, calculating intensity gradients, non-maximum suppression, double thresholding, and edge tracking by hysteresis.
Because of these multiple processing steps, the Canny edge detector can reduce noise and produce more accurate and continuous edges compared to basic operators.

7. Comparison of Operators:

Operator	Type	Noise Sensitivity	Edge Quality
Sobel	First-order	Moderate	Good and relatively smooth
Prewitt	First-order	Moderate to high	Simple and clear
Roberts	First-order	High	Fast but less smooth
Canny	Multi-step	Low	Thin, clean, and accurate

By applying these operators to the same image and comparing the outputs, we can observe that Canny generally produces the cleanest and most well-defined edges, while Sobel and Prewitt provide simpler gradient-based edge detection. Roberts is computationally simple but more sensitive to noise.

Source code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image = cv2.imread("cameraman.jpg", cv2.IMREAD_GRAYSCALE)
# 2. Sobel Edge Detection
sobel_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobel_x, sobel_y)
sobel = cv2.convertScaleAbs(sobel)
```

```
# 3. Prewitt Edge Detection
prewitt_x = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
])
prewitt_y = np.array([
    [-1, -1, -1],
    [ 0,  0,  0],
    [ 1,  1,  1]
])
px = cv2.filter2D(image, cv2.CV_64F, prewitt_x)
py = cv2.filter2D(image, cv2.CV_64F, prewitt_y)

prewitt = cv2.magnitude(px, py)
prewitt = cv2.convertScaleAbs(prewitt)

# 4. Roberts Edge Detection
roberts_x = np.array([
    [1, 0],
    [0, -1]
])
roberts_y = np.array([
    [0, 1],
    [-1, 0]
])
rx = cv2.filter2D(image, cv2.CV_64F, roberts_x)
ry = cv2.filter2D(image, cv2.CV_64F, roberts_y)
roberts = cv2.magnitude(rx, ry)
roberts = cv2.convertScaleAbs(roberts)

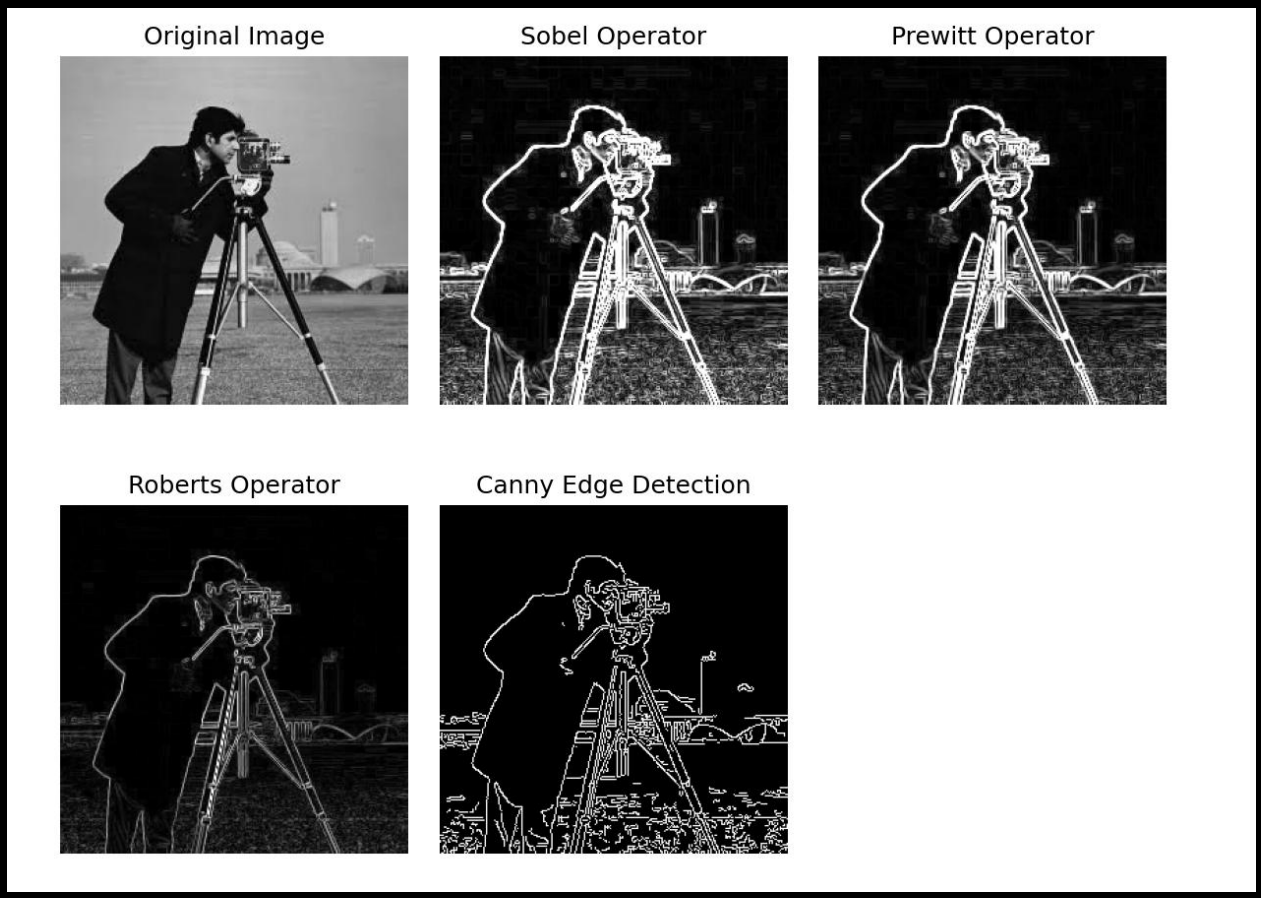
# 5. Canny Edge Detection
canny = cv2.Canny(image, 100, 200)
# 6. Display and Compare
plt.figure(figsize=(12, 10))

plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 3, 2)
plt.imshow(sobel, cmap="gray")
```

```
plt.title("Sobel Operator")
plt.axis("off")
plt.subplot(2, 3, 3)
plt.imshow(prewitt, cmap="gray")
plt.title("Prewitt Operator")
plt.axis("off")
plt.subplot(2, 3, 4)
plt.imshow(roberts, cmap="gray")
plt.title("Roberts Operator")
plt.axis("off")
plt.subplot(2, 3, 5)
plt.imshow(canny, cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:



Result:

Edge detection was successfully performed using Sobel, Prewitt, Roberts, and Canny operators. The edge images obtained from the different operators were displayed and compared with the original image.

Discussion:

Different operators produce different edge detection results. Roberts detects sharp edges but is more sensitive to noise. Prewitt and Sobel provide better edge detection using 3×3 masks, with Sobel generally producing smoother results. Canny provides more clear, thin, and continuous edges and usually gives the best overall edge detection result.

Conclusion:

Edge detection was successfully performed using different operators in Python. The experiment demonstrated that Sobel, Prewitt, Roberts, and Canny can be used to detect object boundaries and important details. Among them, Canny generally provides a cleaner and more well-defined edge image.

Experiment No.: 07

Name of the Experiment: To Write a Python Program to Read 'coins.png' Image, Label All Coins, and Display the Area of All Coins.

Objectives:

1. To understand the concept of image segmentation and object detection in image processing.
2. To learn how to separate coins from the background using thresholding.
3. To detect the boundary of each coin using contour detection.
4. To calculate the area of each coin and mark/label each coin on the image.
5. To display the total number of coins and the area of every coin.

Theory:

1. Digital Image:
A digital image is represented as a two-dimensional function f(x, y), where x and y are the position (coordinates) of a point in the image, and the value of f at that point is called the intensity or gray level. In a grayscale image, this intensity value usually ranges from 0 to 255, where 0 means black, 255 means white, and the values in between represent different shades of gray.

2. Grayscale Conversion:
The color image is first converted into a grayscale image, because color information is not needed to detect the shape and size of coins. This reduces the amount of data and makes processing simpler and faster.

3. Gaussian Blur (Noise Reduction):
A Gaussian filter is a low pass filter that smooths the image by taking a weighted average of the neighboring pixels, where pixels closer to the center get more weight. In this experiment, a 7×7 Gaussian kernel is used to reduce noise and small unwanted details, so that the coin edges can be detected more clearly and no false objects are created during thresholding.

4. Thresholding (Otsu's Method):
Thresholding converts a grayscale image into a binary image, where each pixel becomes either white (255) or black (0). Otsu's method is an automatic thresholding technique that selects the best threshold value by minimizing the variation of pixel intensities within the object group and the background group. In this experiment, THRESH_BINARY_INV is used, so the coins become white and the background becomes black, which makes the coins easier to detect.

5. Contour Detection:
A contour is a curve that joins all the continuous points along the boundary of a connected white region in a binary image. In this experiment, each coin forms one connected white region, so each coin gives one contour. The function cv2.findContours() is used with the RETR_EXTERNAL mode, which finds only the outer boundaries of objects and ignores any inner details (like patterns or holes inside the coin).

6. Area Calculation:
The area of each coin is calculated using cv2.contourArea(), which finds the area enclosed by the contour of the coin. This gives the actual size of the coin in pixels, and it does not assume that the coin is a perfect circle. Objects with a very small area (less than 5000 pixels in this experiment) are treated as noise and removed, so that only real coins are counted.

7. Minimum Enclosing Circle and Labeling:
The function cv2.minEnclosingCircle() finds the smallest circle that completely covers a contour. It gives the center (x, y) and radius of each coin, which is used to draw a circle around the coin and to write the coin number and its area on the image. This is how each coin is labeled.

8. Working Principle:

- The coin image is read and converted into grayscale.
- Gaussian blur is applied to reduce noise.
- Otsu's thresholding converts the image into a binary image where coins are white.
- Contours are detected from the binary image.
- The area of each contour is calculated, and very small objects are removed.
- The coins are sorted from top to bottom and left to right, and each coin is numbered.
- A circle, center point, coin number, and area are drawn on the output image, and the areas are printed.

9. Applications:

- Automatic counting of coins in vending machines and banking systems
- Counting and measuring objects such as seeds, cells, and tablets
- Quality inspection in industries
- Size-based sorting of objects

Source code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("coins.png")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Blur image
gray = cv2.GaussianBlur(gray, (7, 7), 0)

# Convert to binary using Otsu thresholding
_, binary = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Find contours
contours, _ = cv2.findContours(
    binary,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

# Copy image
result = img.copy()

coins = []

# Find coin contours
for contour in contours:

    # Calculate actual area of coin
    area = cv2.contourArea(contour)
```

```
    # Remove very small objects
    if area > 5000:

        # Find bounding circle
        (x, y), radius = cv2.minEnclosingCircle(contour)

        x = int(x)
        y = int(y)
        radius = int(radius)

        coins.append((x, y, radius, area))

# Sort coins from top to bottom and left to right
coins.sort(key=lambda c: (c[1], c[0]))

print("=======================================")
print("         COIN MEASUREMENTS")
print("=======================================")

print("Number of coins:", len(coins))
print()

print("Area of each coin:")
print("--------------------------")

# Display each coin
for i, (x, y, radius, area) in enumerate(coins, 1):

    # Draw circle around coin
    cv2.circle(
        result,
        (x, y),
        radius,
        (0, 255, 0),
        3
    )

    # Draw center
    cv2.circle(
        result,
```

```
        (x, y),
        5,
        (255, 0, 0),
        -1
    )

    # Display coin number and area
    text = f"Coin {i}: {area:.0f} px"

    cv2.putText(
        result,
        text,
        (x - 70, y),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 0, 255),
        2
    )

    print(f"Coin {i}: Area = {area:.2f} pixels")


# Convert BGR to RGB for displaying with Matplotlib
img_rgb = cv2.cvtColor(
    img,
    cv2.COLOR_BGR2RGB
)

result_rgb = cv2.cvtColor(
    result,
    cv2.COLOR_BGR2RGB
)

# Display results
plt.figure(figsize=(18, 6))

# Original image
plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
```

```
plt.axis("off")

# Binary image
plt.subplot(1, 3, 2)
plt.imshow(binary, cmap="gray")
plt.title("Binary Image")
plt.axis("off")

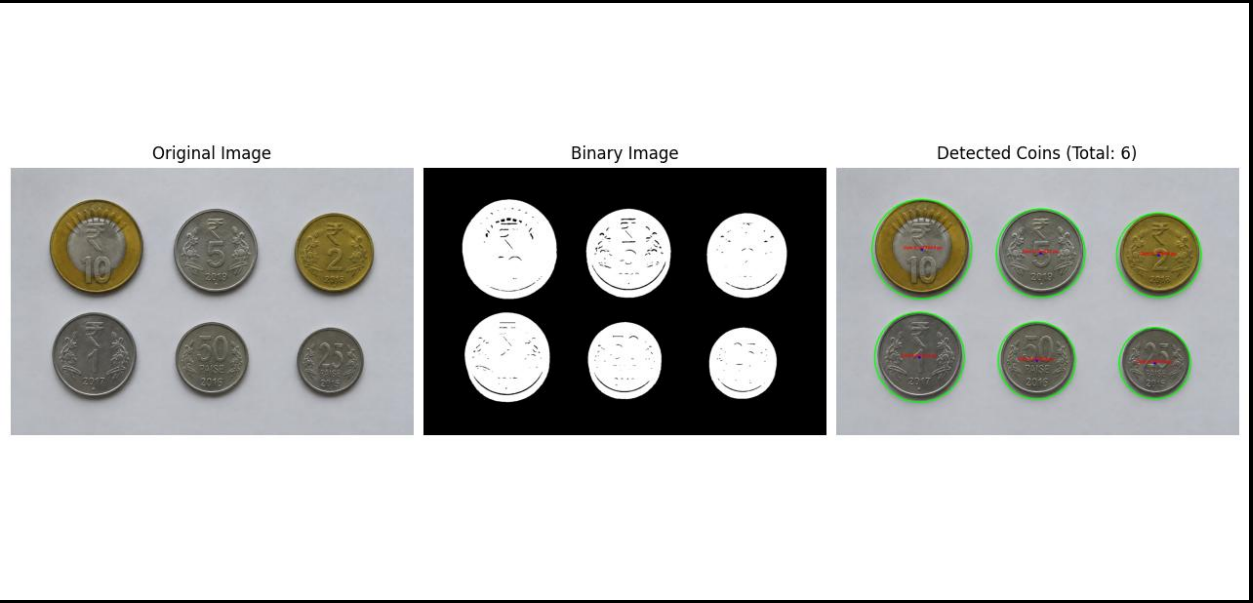
# Detected coins with area
plt.subplot(1, 3, 3)
plt.imshow(result_rgb)
plt.title(f"Detected Coins (Total: {len(coins)})")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:

```
====================================
        COIN MEASUREMENTS
====================================
Number of coins: 6

Area of each coin:
----------------------------
Coin 1: Area = 107693.50 pixels
Coin 2: Area = 86327.00 pixels
Coin 3: Area = 77909.00 pixels
Coin 4: Area = 87514.50 pixels
Coin 5: Area = 65128.00 pixels
Coin 6: Area = 55373.00 pixels
```



Result:

The ‘coins.png’ image was successfully processed using the Python program. The following results were obtained:

- Total number of coins detected: **6**

The output showed the original image, the binary image, and the final image where each coin is marked with a green circle, its center point, its coin number, and its area.

Discussion:

In this experiment, the coins.png image was converted to grayscale, smoothed using a Gaussian filter, and converted into a binary image using Otsu’s thresholding. Contours were detected, and the area of each coin was calculated. Objects smaller than 5000 pixels were removed as noise. Each coin was labeled with a circle, number, and area. Actual contour areas helped distinguish coins of different sizes. However, touching coins may be detected as one object, and uneven lighting or shadows may affect thresholding. Watershed segmentation and adaptive thresholding can reduce these problems.

Conclusion:

The experiment successfully detected, counted, labeled, and measured coins using thresholding and contour detection. This method is simple and effective when coins are clearly separated from the background.

Experiment No.: 08
Name of the Experiment: To display image operations in Python – i) Threshold Image ii) Power-law Enhanced Image iii) High Pass Image.

Objectives:

1. To learn how to read and display an image in Python using OpenCV.
2. To convert a color image into a grayscale image for image processing.
3. To apply thresholding to convert a grayscale image into a binary image.
4. To apply power-law transformation to enhance the intensity of an image.
5. To apply a high pass filter to highlight edges and fine details of an image.
6. To compare the original image with the processed images and understand the effect of each operation.

Theory:

Digital Image:
A digital image is represented as a two-dimensional function f(x, y), where x and y are the position (coordinates) of a point in the image, and the value of f at that point is called the intensity or gray level. In a grayscale image, this intensity value usually ranges from 0 to 255, where 0 means black, 255 means white, and the values in between represent different shades of gray.

Thresholding:
Thresholding is a simple and widely used image segmentation technique that converts a grayscale image into a binary image. In this method, a threshold value (T) is chosen, and each pixel of the image is compared with this value:

- If the pixel intensity is greater than T, the pixel is set to white (255).
- If the pixel intensity is less than or equal to T, the pixel is set to black (0).

Mathematically:

$g(x, y) = 255$, if $f(x, y) > T$
$g(x, y) = 0$, if $f(x, y) \leq T$

Thresholding is mainly used to separate objects (foreground) from the background, and it is a fundamental step in many image processing applications like object detection and segmentation.

Power Law (Gamma) Transformation:
Power law transformation, also known as gamma correction, is used to enhance the contrast and brightness of an image. It is defined by the equation:

$s = c \cdot r ^ \gamma$

where:

- r is the input pixel intensity
- s is the output pixel intensity
- c is a constant (usually 1)
- γ (gamma) is a positive value that controls the transformation
- If $\gamma < 1$, the transformation enhances (brightens) the dark regions of the image, improving contrast in darker areas.
- If $\gamma > 1$, the transformation enhances the bright regions and darkens the darker areas, improving contrast in brighter areas.
- If $\gamma = 1$, the image remains unchanged.

This technique is widely used to correct the brightness and contrast of images, especially those that are too dark or too bright due to lighting conditions or display/capture devices.

High Pass Filter (HPF):
A high pass filter allows high-frequency components of an image to pass through while removing or reducing the low-frequency components. Since edges, fine details, and noise correspond to high-frequency information, high pass filters are mainly used for:

- Edge detection
- Image sharpening

A commonly used high pass filter kernel is:

```
-1  -1  -1
-1   8  -1
-1  -1  -1
```

This kernel works by comparing each pixel with its 8 surrounding neighbors. Since the sum of all values in the kernel is zero, it removes the flat/smooth areas of the image and keeps only the regions with sharp intensity changes, resulting in an edge-highlighted output image.

Relationship Between the Three Operations:

- Thresholding simplifies the image into two distinct regions (object and background) based on intensity.
- Power law transformation improves the visibility of details by adjusting brightness/contrast without changing the structure of the image.

- High pass filtering highlights the edges and fine details by removing the smooth, low-frequency areas.

Together, these three operations demonstrate different fundamental approaches to image enhancement and segmentation — intensity-based segmentation, contrast enhancement, and frequency-based detail extraction.

Source code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read original image
img_color = cv2.imread("cameraman.jpg")

# Convert to grayscale
img_gray = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)

# i) Threshold Image
_, threshold = cv2.threshold(img_gray, 127, 255, cv2.THRESH_BINARY)

# ii) Power-law Enhancement
gamma = 0.5

normalized = img_gray / 255.0
power_img = np.power(normalized, gamma)
power_img = np.uint8(power_img * 255)

# iii) High-pass Image
blur = cv2.GaussianBlur(img_gray, (5, 5), 0)
high_pass = cv2.subtract(img_gray, blur)

# Display
plt.figure(figsize=(12, 8))

# Original
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(img_color, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
```

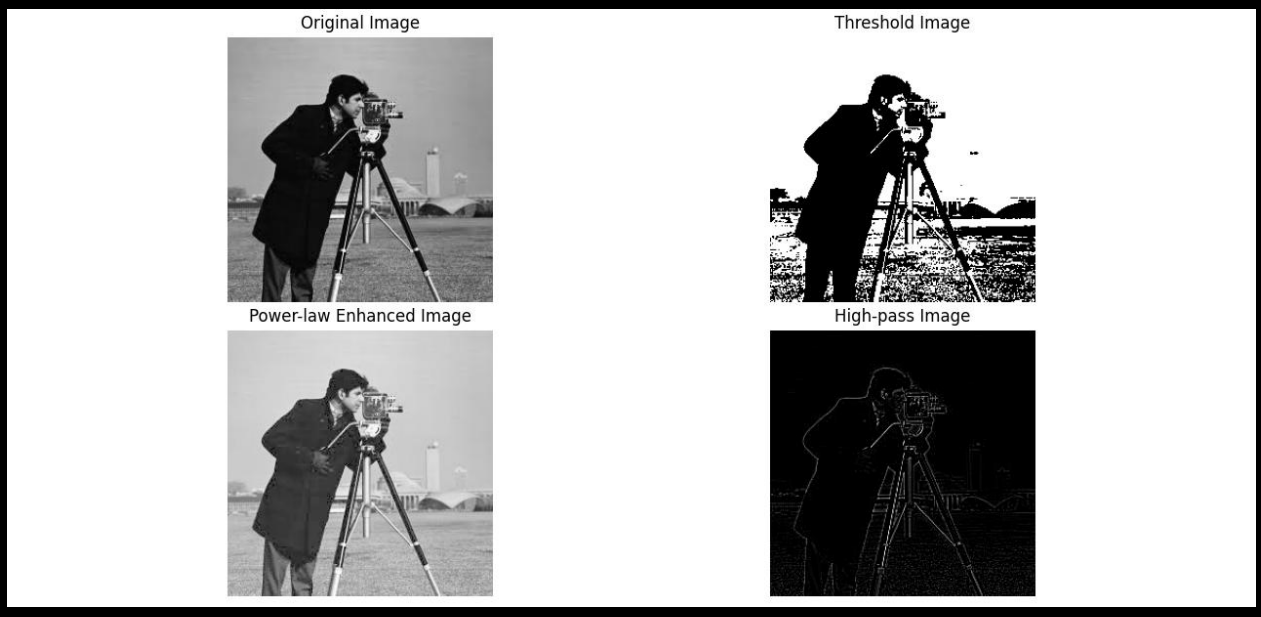
```
# Threshold
plt.subplot(2, 2, 2)
plt.imshow(threshold, cmap="gray")
plt.title("Threshold Image")
plt.axis("off")

# Power-law
plt.subplot(2, 2, 3)
plt.imshow(power_img, cmap="gray")
plt.title("Power-law Enhanced Image")
plt.axis("off")

# High-pass
plt.subplot(2, 2, 4)
plt.imshow(high_pass, cmap="gray")
plt.title("High-pass Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:



Result:
Thresholding, power-law enhancement, and high-pass filtering were successfully performed in Python. The threshold, power-law enhanced, and high-pass images were displayed and compared with the original image.

Discussion:
Different image operations produce different results. Thresholding converts the grayscale image into a binary image based on a selected threshold value. Power-law enhancement with $\gamma = 0.5$ improves the brightness of darker regions and makes image details more visible. High-pass filtering reduces smooth regions and highlights edges, fine details, and sudden changes in intensity.

Conclusion:
The required image operations were successfully performed using Python. The experiment demonstrated that thresholding can be used for image segmentation, power-law transformation can be used for intensity enhancement, and high-pass filtering can be used to highlight edges and fine details of an image.

Experiment No.: 09

Name of the Experiment:

To perform image enhancement, smoothing and sharpening in spatial domain using different spatial filters and compare their performances.

Objectives:

1. To perform image enhancement using different spatial domain techniques.
2. To improve the contrast of an image using Histogram Equalization and CLAHE.
3. To apply different smoothing filters such as Mean, Gaussian, and Median filters.
4. To perform image sharpening using Laplacian and Unsharp Masking techniques.
5. To compare the effects of different spatial filters on an image.
6. To understand the applications of spatial filtering in image processing.

Theory:

1. Spatial Domain Image Processing:

Spatial domain image processing refers to directly manipulating the pixels of an image. In spatial domain techniques, the value of a pixel is changed based on its original value and the values of its neighboring pixels.

A general spatial domain operation can be represented as:

$g(x, y) = T[f(x, y)]$

where,

$f(x, y)$ = input image

$g(x, y)$ = processed image

T = transformation or operation applied to the image.

Spatial domain techniques are commonly used for image enhancement, smoothing, noise reduction, sharpening, and edge detection**.**

2. Image Enhancement:

Image enhancement is the process of improving the visual quality of an image by increasing its contrast, brightness, or visibility of important details.

In this experiment, two image enhancement techniques are used:

a) Histogram Equalization:

Histogram Equalization is a technique used to improve the contrast of an image by redistributing its intensity values.

It attempts to spread the pixel intensity values over a wider range from 0 to 255.

A low-contrast image may have most of its pixels concentrated within a small intensity range. Histogram equalization redistributes these values to produce better contrast.

It is especially useful when the image has poor overall contrast.

b) CLAHE:
CLAHE stands for Contrast Limited Adaptive Histogram Equalization.
Unlike ordinary histogram equalization, CLAHE works on small regions or tiles of an image instead of processing the entire image at once.
The image is divided into small regions, and histogram equalization is performed separately in each region.
The clip limit prevents excessive contrast enhancement and reduces noise amplification.
CLAHE is useful for enhancing local details, especially in images where different regions have different brightness levels.

3. Image Smoothing:
Image smoothing is used to reduce noise and unwanted small details from an image. It generally reduces high-frequency components and makes the image appear smoother.
In this experiment, three smoothing filters are used:
a) Mean Filter:
The Mean Filter, also called the Average Filter, replaces each pixel with the average value of its neighboring pixels.
A commonly used 3 × 3 mean filter is:

$$\frac{1}{9} \times \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix}$$

It smooths the image by averaging neighboring pixels. However, it can also blur edges.
In Python, the Mean Filter is applied using:
cv2.blur(gray, (5, 5))

b) Gaussian Filter:
The Gaussian Filter is a smoothing filter that uses a Gaussian function to calculate weighted averages of neighboring pixels.
Pixels closer to the center receive higher weights than pixels farther away.
Gaussian filtering provides smooth results and is commonly used for noise reduction and preprocessing before edge detection.
In Python:
cv2.GaussianBlur(gray, (5, 5), 0)

c) Median Filter:
The Median Filter replaces the value of a pixel with the median value of its neighboring pixels.
For example, if the neighboring pixel values are:
10, 12, 15, 18, 200

The median value is 15, so the extreme value 200 is removed.
Therefore, the Median Filter is particularly effective for removing salt-and-pepper noise while preserving edges better than the Mean Filter.
In Python:
cv2.medianBlur(gray, 5)

4. Image Sharpening:
Image sharpening is used to enhance edges, fine details, and sudden changes in intensity in an image.
Sharpening increases the visual clarity of an image by emphasizing high-frequency components.
In this experiment, two sharpening techniques are used.
a) Laplacian Sharpening:
The Laplacian operator is a second-order derivative operator that detects regions of rapid intensity changes.
A commonly used Laplacian mask is:
 0 -1 0
-1 4 -1
 0 -1 0
The Laplacian image can be used to enhance the edges and fine details of the original image.
The sharpening operation can be represented as:
Sharpened Image = Original Image + Laplacian Image
Laplacian sharpening is useful for emphasizing edges and fine details.

b) Unsharp Masking:
Unsharp Masking is a sharpening technique in which a blurred version of the image is subtracted from the original image to obtain the detail information.
First, the image is blurred using a Gaussian filter.
Then, the sharpened image is obtained by combining the original image with the blurred image.
The basic idea can be represented as:
Mask = Original Image − Blurred Image
Then,
Sharpened Image = Original Image + k × Mask
where k controls the amount of sharpening.
Unsharp Masking enhances edges and fine details while maintaining the overall structure of the image.

5. Comparison of Spatial Filters:
Different spatial filters produce different effects on an image.

Technique	Main Purpose	Effect
Histogram Equalization	Contrast enhancement	Improves global contrast
CLAHE	Local contrast enhancement	Enhances local details
Mean Filter	Smoothing	Reduces noise but may blur edges
Gaussian Filter	Smoothing	Produces smooth and natural blur
Median Filter	Noise removal	Removes salt-and-pepper noise while preserving edges
Laplacian Sharpening	Sharpening	Enhances edges and fine details
Unsharp Masking	Sharpening	Improves image sharpness and detail

Therefore, the best filter depends on the type of image and the required application.

Source code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read Image

img = cv2.imread("cameraman.jpg")

if img is None:
    print("Error: Image not found. Check the file path.")
    exit()

# Convert BGR to RGB for display
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. IMAGE ENHANCEMENT

# Histogram Equalization
hist_eq = cv2.equalizeHist(gray)
```

```
# CLAHE (Contrast Limited Adaptive Histogram Equalization)
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

clahe_img = clahe.apply(gray)

# 2. IMAGE SMOOTHING

# Mean / Average Filter
mean_filter = cv2.blur(gray, (5, 5))

# Gaussian Filter
gaussian_filter = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)

# Median Filter
median_filter = cv2.medianBlur(
    gray,
    5
)

# 3. IMAGE SHARPENING

# ---------- Laplacian Sharpening ----------

laplacian = cv2.Laplacian(
    gray,
    cv2.CV_64F
)

laplacian = np.uint8(
    np.absolute(laplacian)
)

laplacian_sharp = cv2.add(
    gray,
    laplacian
)
```

```
# ---------- Unsharp Masking ----------

blurred = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)

# Unsharp masking
unsharp = cv2.addWeighted(
    gray,
    1.5,
    blurred,
    -0.5,
    0
)

# DISPLAY RESULTS

plt.figure(figsize=(14, 12))

# Original
plt.subplot(3, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# Histogram Equalization
plt.subplot(3, 3, 2)
plt.imshow(hist_eq, cmap="gray")
plt.title("Histogram Equalization")
plt.axis("off")

# CLAHE
plt.subplot(3, 3, 3)
plt.imshow(clahe_img, cmap="gray")
plt.title("CLAHE")
plt.axis("off")

# Mean Filter
plt.subplot(3, 3, 4)
plt.imshow(mean_filter, cmap="gray")
```

```
plt.title("Mean Filter")
plt.axis("off")

# Gaussian Filter
plt.subplot(3, 3, 5)
plt.imshow(gaussian_filter, cmap="gray")
plt.title("Gaussian Filter")
plt.axis("off")

# Median Filter
plt.subplot(3, 3, 6)
plt.imshow(median_filter, cmap="gray")
plt.title("Median Filter")
plt.axis("off")

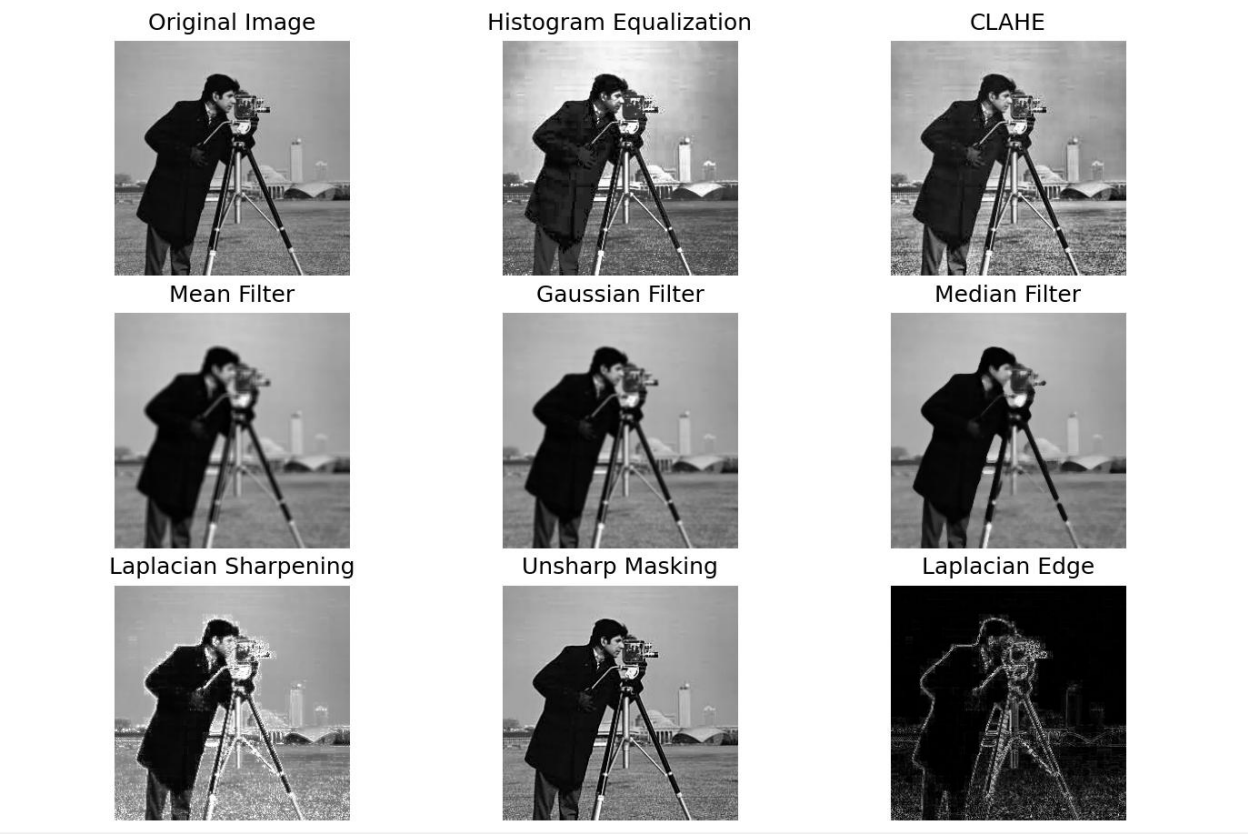
# Laplacian Sharpening
plt.subplot(3, 3, 7)
plt.imshow(laplacian_sharp, cmap="gray")
plt.title("Laplacian Sharpening")
plt.axis("off")

# Unsharp Masking
plt.subplot(3, 3, 8)
plt.imshow(unsharp, cmap="gray")
plt.title("Unsharp Masking")
plt.axis("off")

# Edge Image
plt.subplot(3, 3, 9)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Edge")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:



Result:
Image enhancement, smoothing, and sharpening were successfully performed using different spatial filters. The processed images were displayed and compared with the original image.

Discussion:
Histogram Equalization and CLAHE improved image contrast. Mean, Gaussian, and Median filters smoothed the image and reduced noise. Laplacian and Unsharp Masking enhanced edges and fine details.

Conclusion:
The experiment successfully demonstrated image enhancement, smoothing, and sharpening using spatial filters in Python. Different filters produced different effects depending on their applications.

Experiment No.: 10

Name of the Experiment: To Write a Python Program to Separate Voiced, Unvoiced, and Silence Regions from a Speech Signal.

Objectives:

1. To understand the characteristics of voiced, unvoiced, and silence regions in a speech signal.
2. To understand short-time analysis (framing) of a speech signal.
3. To calculate the short-time energy and zero crossing rate of a speech signal.
4. To classify each part of the speech signal as voiced, unvoiced, or silence using threshold values.
5. To plot and observe the separated regions of the speech signal.

Theory:

1. Speech Signal:
Speech is a continuous, non-stationary signal produced by the human vocal tract. Non-stationary means its properties change rapidly with time. In a computer, it is stored as a digital signal by sampling it at a fixed sampling frequency (fs).

2. Types of Speech Regions:
A speech signal can be divided into three types of regions:

- **Voiced:** Produced when the vocal cords vibrate (for example, vowels like "a", "e", "o"). These regions have high energy, a periodic waveform, and a low zero crossing rate.
- **Unvoiced:** Produced without vocal cord vibration, by forcing air through a narrow gap (for example, "s", "sh", "f"). These regions are noise-like, with lower energy and a high zero crossing rate.
- **Silence:** The parts where there is no speech, only background noise or pauses. These regions have very low energy.

3. Short-Time Analysis (Framing):
Because speech changes quickly, it is not analyzed as a whole. Instead, it is divided into small overlapping segments called frames, usually 20-30 ms long. Within such a short time, the speech signal is assumed to be almost stationary, so its properties can be measured reliably. The frames are moved forward by a smaller step called the hop size (for example, 10 ms), which creates overlap between neighboring frames.

4. Short-Time Energy (STE):
The short-time energy of a frame measures the strength of the signal in that frame:

$$E = \Sigma x^2(n)$$

where x(n) are the samples of the frame. Voiced speech has the highest energy, unvoiced speech has lower energy, and silence has very low energy. Therefore, energy is mainly used to separate silence from speech.

5. Zero Crossing Rate (ZCR):
The zero crossing rate is the rate at which the signal changes its sign (crosses the zero amplitude line) within a frame:

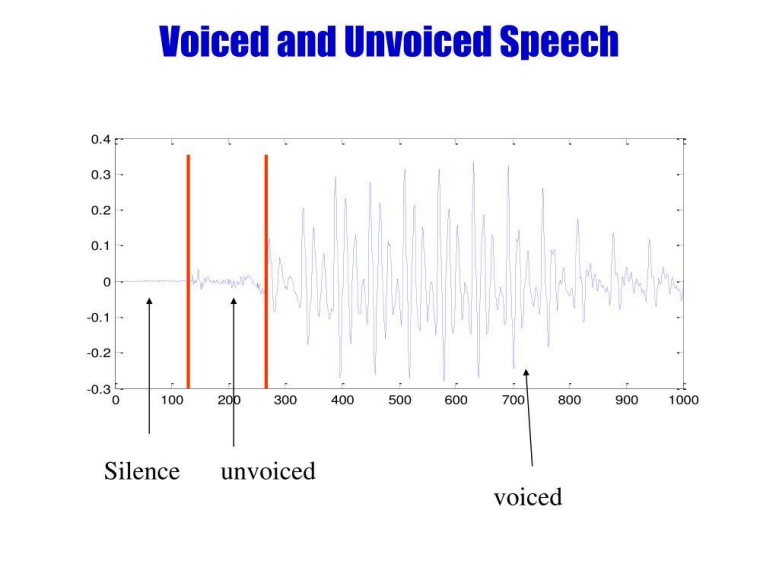
$$ZCR = (1/2N) \Sigma |sgn[x(n)] - sgn[x(n-1)]|$$

where N is the number of samples in the frame. A low ZCR means the signal changes slowly, which is typical of voiced speech (low-frequency periodic signal). A high ZCR means the signal changes rapidly, which is typical of unvoiced speech (high-frequency noise-like signal).

6. Threshold-Based Classification:
Each frame is classified using an energy threshold and a ZCR threshold:

Region	Energy	Zero Crossing Rate
Silence	Very low (below energy threshold)	Not considered
Unvoiced	Above energy threshold	High (above ZCR threshold)
Voiced	Above energy threshold	Low (below ZCR threshold)

The threshold values depend on the recording quality and noise level, so they are usually adjusted for each signal.



7. Applications:

- Speech recognition systems
- Speech coding and compression
- Pitch detection and speech analysis
- Removing silence and noise from speech recordings

Source code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile

# ---------- 1. Read the signal ----------
fs, x = wavfile.read('audio.wav')
if x.ndim > 1:                    # stereo -> mono
    x = x.mean(axis=1)
x = x.astype(float)
x = x / np.max(np.abs(x))         # normalise to [-1, 1]
t = np.arange(len(x)) / fs

# ---------- 2. Framing (25 ms frame, 10 ms hop) ----------
frame_len = int(0.025 * fs)
hop = int(0.010 * fs)
n_frames = 1 + (len(x) - frame_len) // hop

energy = np.zeros(n_frames)
zcr = np.zeros(n_frames)
```

```
for i in range(n_frames):
    seg = x[i * hop: i * hop + frame_len]
    energy[i] = np.sum(seg ** 2)                        # short-time energy
    zcr[i] = 0.5 * np.mean(np.abs(np.diff(np.sign(seg))))  # zero-crossing rate

# ---------- 3. Thresholds (tune these for your recording) ----------
E_thr = 0.02 * np.max(energy)       # below this  -> silence
Z_thr = 0.15                        # above this  -> unvoiced, else voiced

# ---------- 4. Classify each frame: 0 = silence, 1 = unvoiced, 2 = voiced ----------
label = np.zeros(n_frames, dtype=int)
for i in range(n_frames):
    if energy[i] < E_thr:
        label[i] = 0
    elif zcr[i] > Z_thr:
        label[i] = 1
    else:
        label[i] = 2

# ---------- 5. Expand frame labels to sample level ----------
sample_label = np.zeros(len(x), dtype=int)
for i in range(n_frames):
    sample_label[i * hop: i * hop + hop] = label[i]
sample_label[n_frames * hop:] = label[-1]

# ---------- 6. Plots ----------
names = ['Silence', 'Unvoiced', 'Voiced']
colors = ['gray', 'red', 'blue']

plt.figure(figsize=(11, 8))

plt.subplot(4, 1, 1)
plt.plot(t, x, 'k')
plt.title('Original speech signal'); plt.ylabel('Amplitude')

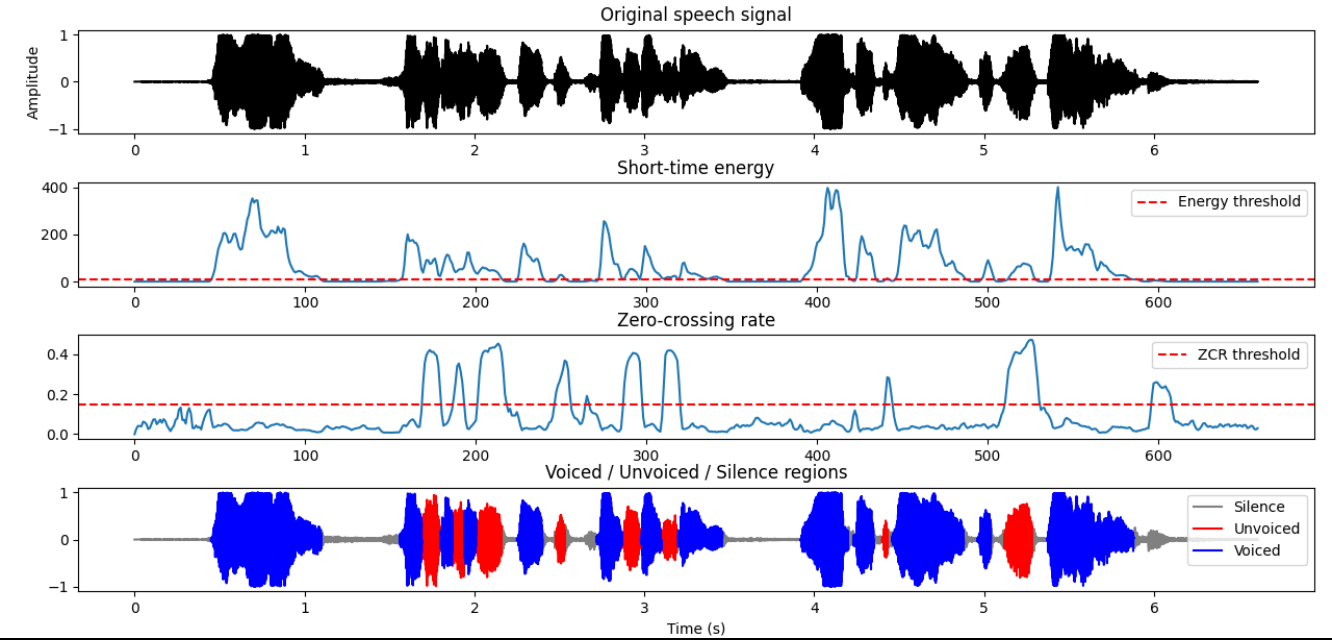
plt.subplot(4, 1, 2)
plt.plot(energy); plt.axhline(E_thr, color='r', ls='--', label='Energy threshold')
plt.title('Short-time energy'); plt.legend()

plt.subplot(4, 1, 3)
plt.plot(zcr); plt.axhline(Z_thr, color='r', ls='--', label='ZCR threshold')
plt.title('Zero-crossing rate'); plt.legend()
```

```
plt.subplot(4, 1, 4)
for c in range(3):
    y = np.where(sample_label == c, x, np.nan)
    plt.plot(t, y, color=colors[c], label=names[c])
plt.title('Voiced / Unvoiced / Silence regions')
plt.xlabel('Time (s)'); plt.legend(loc='upper right')

plt.tight_layout()
plt.show()
```

Output:



Result:

The speech signal was divided into 25 ms frames with a hop size of 10 ms. Short-time energy and zero crossing rate (ZCR) were calculated to classify the frames into voiced, unvoiced, and silence regions. The original signal, energy, ZCR, and classified speech regions were plotted successfully.

Parameter	Value
Frame length	25 ms
Hop size	10 ms
Energy threshold	2% of maximum energy
ZCR threshold	0.15

Discussion

In this experiment, the speech signal was divided into short frames, and short-time energy and zero crossing rate (ZCR) were calculated.

- Silence: Frames with low energy.
- Voiced: Frames with high energy and low ZCR.
- Unvoiced: Frames with high ZCR.

The method can identify speech and silence, but background noise and threshold selection may cause some classification errors.

Conclusion

The experiment successfully classified speech into voiced, unvoiced, and silence regions using short-time energy and ZCR. This method is useful for basic speech signal analysis.

Experiment No.: 11

Name of the Experiment: To Write a Python Program and Plot Multilevel Speech Resolution (Multiresolution Analysis of a Speech Signal Using Wavelet Decomposition).

Objectives:

1. To understand the concept of multilevel (multiresolution) analysis of a speech signal.
2. To understand the basic idea of the wavelet transform and why it is useful for speech signals.
3. To decompose a speech signal into approximation and detail components at multiple levels.
4. To plot the components of each level and observe their frequency bands.
5. To verify that the original signal can be reconstructed from the decomposed components.

Theory:

1. Limitation of the Fourier Transform:
The Fourier Transform shows which frequencies are present in a signal, but it does not show when they occur. Speech and audio signals are non-stationary, meaning their frequency content changes with time, so a method that gives both time and frequency information is needed.

2. Wavelet Transform:
The wavelet transform analyzes a signal using small wave-like functions called wavelets, which are stretched (scaled) and shifted along the signal. Narrow wavelets capture fast changes (high frequencies) and wide wavelets capture slow changes (low frequencies). This gives a good time-frequency representation of the signal.

3. Discrete Wavelet Transform (DWT):
In the DWT, the signal is passed through a pair of filters:

- **Low pass filter:** gives the approximation coefficients (cA), which represent the low-frequency, smooth part of the signal.
- **High pass filter:** gives the detail coefficients (cD), which represent the high-frequency part, such as rapid changes and noise.

After filtering, each output is downsampled by 2.

4. Multilevel Decomposition (Multiresolution Analysis):
At each level, only the approximation coefficients are decomposed again into a new approximation and a new detail. Each level splits the frequency band into two halves, so the signal is analyzed at different resolutions. For an N-level decomposition, the signal is represented as A_N, D_N, D_(N-1), ..., D_1.

For sampling frequency fs, the frequency bands are:

Component	Frequency Band
D1	fs/4 to fs/2
D2	fs/8 to fs/4
D3	fs/16 to fs/8
...	...
D_N	fs/2^(N+1) to fs/2^N
A_N	0 to fs/2^(N+1)

5. Wavelet Families:
There are many types of wavelets, such as Haar, Daubechies, Symlets, and Coiflets. A wavelet is chosen based on the type of signal. Daubechies wavelets are orthogonal and have compact support, so they are widely used for speech and audio analysis.

6. Time and Frequency Resolution:
At low frequencies (higher levels), the frequency resolution is good but the time resolution is poor. At high frequencies (lower levels), the time resolution is good but the frequency resolution is poor. This is the main advantage of the wavelet transform over the Fourier Transform for non-stationary signals.

7. Reconstruction:
The original signal can be recovered from its components using the Inverse DWT. Since the wavelet transform is orthogonal, no information is lost, and the sum of all the components gives back the original signal:

$x(t) = A_N + D_N + D_{(N-1)} + ... + D_1$

The small difference between the original and reconstructed signal is called the reconstruction error, which should be nearly zero.

8. Applications:

- Noise removal (denoising) of audio and speech signals
- Audio and image compression

- Speech analysis and feature extraction
- ECG/EEG and other biomedical signal analysis

Source code:

```
import numpy as np
import matplotlib.pyplot as plt
import pywt
from scipy.io import wavfile

# ---------- 1. Read the signal ----------
fs, x = wavfile.read('audio.wav')
if x.ndim > 1:
    x = x.mean(axis=1)
x = x.astype(float)
x = x / np.max(np.abs(x))
t = np.arange(len(x)) / fs

wavelet = 'db4'
levels = 4

# ---------- 2. Multilevel decomposition ----------
# coeffs = [cA4, cD4, cD3, cD2, cD1]
coeffs = pywt.wavedec(x, wavelet, level=levels)


def reconstruct(keep_index):
    """Rebuild a full-length signal using only ONE coefficient set."""
    c = [np.zeros_like(a) for a in coeffs]
    c[keep_index] = coeffs[keep_index]
    return pywt.waverec(c, wavelet)[:len(x)]


# Approximation at the deepest level and details at every level
approx = reconstruct(0)                                  # A4
details = {k: reconstruct(levels - k + 1) for k in range(1, levels + 1)}  # D1..D4

# Approximations at intermediate levels A1..A3 (decompose to that level, keep
cA)
```

```
approximations = {}
for k in range(1, levels + 1):
    ck = pywt.wavedec(x, wavelet, level=k)
    ck = [ck[0]] + [np.zeros_like(a) for a in ck[1:]]
    approximations[k] = pywt.waverec(ck, wavelet)[:len(x)]

# ---------- 3. Plot ----------
plt.figure(figsize=(11, 12))

plt.subplot(levels + 1, 2, 1)
plt.plot(t, x, 'k'); plt.title('Original signal')

for k in range(1, levels + 1):
    plt.subplot(levels + 1, 2, 2 * k + 1)
    plt.plot(t, approximations[k], 'b')
    plt.title(f'Approximation A{k}  (0 - {fs / 2 ** (k + 1):.0f} Hz)')

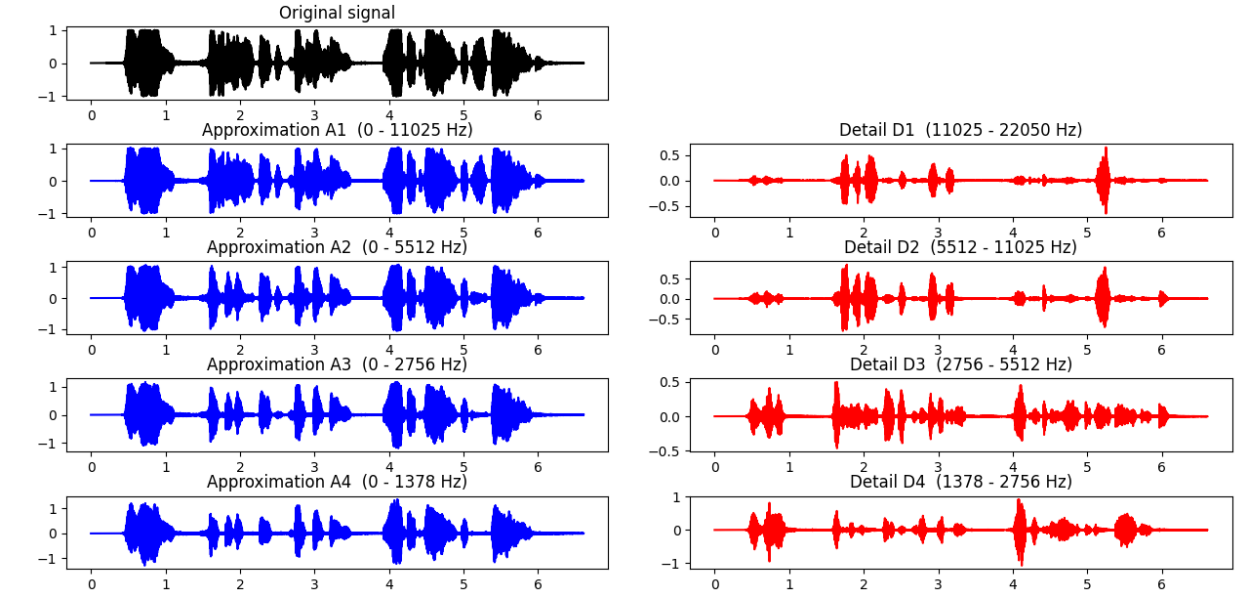
    plt.subplot(levels + 1, 2, 2 * k + 2)
    plt.plot(t, details[k], 'r')
    plt.title(f'Detail D{k}  ({fs / 2 ** (k + 1):.0f} - {fs / 2 ** k:.0f} Hz)')

plt.xlabel('Time (s)')
plt.tight_layout()
plt.show()

# ---------- 4. Check: A4 + D4 + D3 + D2 + D1 gives back the original ----------
recon = approx + sum(details.values())
print('Max reconstruction error:', np.max(np.abs(x - recon)))
```

Output:

Max reconstruction error: 9.992007221626409e-16



Result:

The audio signal audio.wav with a sampling frequency of 44100 Hz was decomposed into 4 levels using the db4 wavelet. The signal duration was approximately 6.6 seconds.

The original signal showed speech activity from about 0.4 s to 6 s, with silence at the beginning and end. Approximations A1 to A4 became smoother as the level increased. Details D1 to D4 represented different frequency bands. D1 had a small amplitude, while D3 and D4 showed stronger activity during speech.

The maximum reconstruction error was 9.992007221626409e-16.

Component	Frequency Band (Hz)
D1	11025–22050
D2	5512–11025
D3	2756–5512
D4	1378–2756
A4	0–1378

Discussion:

In this experiment, the audio signal was decomposed into 4 levels using the db4 wavelet. The approximations became smoother at higher levels, while the detail components represented different frequency bands. Wavelet transform provides good time-frequency analysis for speech signals. The original signal was reconstructed with a very small error.

Conclusion:

The experiment successfully performed 4-level wavelet decomposition using the db4 wavelet. The signal was separated into approximation and detail components and reconstructed successfully. Wavelet transform is useful for speech analysis, denoising, and compression.

Experiment No.: 12

Name of the Experiment: To Write a Python Program to Recognize a Speech Signal.

Objectives:

1. To understand the basic concept of speech recognition.
2. To read a speech signal from an audio file in Python.
3. To convert the speech signal into text using a speech recognition engine.
4. To handle errors when the speech cannot be understood or the service cannot be reached.

Theory:

1. Speech Signal:
Speech is a continuous, non-stationary signal produced by the human vocal tract. When it is recorded, it is stored as a digital audio signal (for example, a .wav file) by sampling the signal at a fixed sampling frequency and storing each sample as a number.

2. Speech Recognition:
Speech recognition, also called Speech-to-Text (STT), is the process of converting spoken words into written text. It is used in voice assistants, voice typing, automatic subtitles, and voice-controlled systems.

3. Basic Steps of a Speech Recognition System:

- **Audio input:** The speech signal is recorded or read from a file.
- **Preprocessing:** Noise is reduced and the signal is divided into small frames (usually 20-30 ms long).
- **Feature extraction:** Useful features, such as MFCC (Mel-Frequency Cepstral Coefficients), are extracted from each frame.
- **Acoustic model:** It matches the features with basic speech sounds called phonemes.
- **Language model:** It uses grammar and word probabilities to select the most likely sequence of words.
- **Output:** The final recognized text is produced.

4. Online Speech Recognition Service:
In this experiment, recognition is done using the Google Web Speech API, which is an online service. The audio is sent to Google's server, where a trained

model converts it into text, and the text is returned to the program. Because of this, an **internet connection is required**.

5. Factors Affecting Recognition Accuracy:

- Clarity of the voice and speaking speed
- Background noise
- Accent and pronunciation
- Quality of the microphone and recording

6. Exception Handling:
Two types of errors are handled in the program:

- **Unknown value error:** Occurs when the speech is unclear, too noisy, or silent, so it cannot be understood.
- **Request error:** Occurs when the recognition service cannot be reached, for example due to no internet connection.

7. Working Principle:

- A recognizer object is created.
- The audio file voice.wav is opened and the audio data is read.
- The audio is sent to the Google Web Speech API for recognition.
- The recognized text is printed. If an error occurs, a suitable message is displayed.

8. Applications:

- Voice assistants (Google Assistant, Siri, Alexa)
- Automatic subtitle and caption generation
- Voice typing and dictation
- Voice-controlled devices and security systems

Source Code:

```
import speech_recognition as sr

# Create recognizer object
recognizer = sr.Recognizer()

# Load speech file
with sr.AudioFile("voice.wav") as source:
    audio = recognizer.record(source)
```

```
# Recognize speech
try:
    result = recognizer.recognize_google(audio)

    print("Recognized Speech:")
    print(result)

except sr.UnknownValueError:
    print("Sorry, speech could not be understood.")

except sr.RequestError:
    print("Could not connect to the recognition service.")
```

Output:

Recognized Speech:
this is a test of me recording my voice.

Result:

The audio file ‘voice.wav’ was successfully processed using the Python program. The following text was recognized from the speech:

Recognized Speech: this is a test of me recording my voice.

Discussion:

In this experiment, a speech signal was converted into text using the Google Web Speech API. The recognition accuracy depended on voice clarity, background noise, accent, and speaking speed. An internet connection was required, and errors were handled when speech could not be recognized or the service was unavailable.

Conclusion:

The experiment successfully converted speech from an audio file into text using Python. It demonstrated the basic working of speech recognition and its applications in voice assistants and automatic transcription.

Experiment No.: 13
Name of the Experiment: To Write a Python Program to Perform Text-to-Speech Conversion and to Record, Save, Play, and Plot a Speech Signal.

Objectives:

1. To understand the basic concepts of speech signals, sampling, and digital audio.
2. To understand how text can be converted into speech (Text-to-Speech).
3. To record a speech signal using a microphone with a given sampling frequency and duration.
4. To save the recorded speech as an audio file and play it back.
5. To plot the speech signal in the time domain and observe how its amplitude changes with time.

Theory:

1. Speech Signal:
Speech is a continuous, non-stationary signal produced by the human vocal tract. To process it in a computer, the analog speech signal is converted into a digital signal by sampling and quantization.

2. Sampling and Sampling Frequency:
Sampling means measuring the amplitude of an analog signal at regular time intervals. The number of samples taken per second is called the sampling frequency (fs). According to the Nyquist theorem, the sampling frequency must be at least twice the highest frequency present in the signal:

$fs \geq 2 \cdot f_max$

Human speech mainly lies below 4-8 kHz, so a sampling frequency such as 44100 Hz (CD quality) is more than enough to capture speech accurately. A higher sampling frequency gives better quality but also produces a larger file size.

3. Text-to-Speech (TTS):
Text-to-Speech is the process of converting written text into spoken voice. A TTS system generally works in the following steps:

- **Text analysis:** The text is cleaned and converted into words and phonemes (basic speech sounds).
- **Prosody generation:** Pitch, rhythm, and duration of the speech are decided.

- **Speech synthesis:** The final speech waveform is generated and played through the speaker.

TTS is used in voice assistants, screen readers, navigation systems, and many other applications.

4. Speech Recording:
To record speech, a microphone converts sound waves into an electrical signal, and the sound card converts this signal into digital samples. The recording is controlled by three main parameters: sampling frequency, duration, and number of channels (mono or stereo). The total number of samples is:

Number of samples = duration × fs

5. Storing and Playing Audio:
A recorded signal can be stored as an audio file, such as WAV. The WAV format stores uncompressed audio, so the quality of the recorded speech is preserved. The stored signal can later be played back through a speaker.

6. Time Domain Representation:
A speech signal can be plotted as amplitude versus time. The time axis is calculated as:

time = sample index / fs

In the plot, silent parts have very small amplitude, and the parts where the person speaks show larger amplitude variation.

7. Applications:

- Voice assistants and screen readers for visually impaired people
- Voice recording and voice-controlled systems
- Speech recognition and speaker identification systems
- Audio editing and speech analysis

Source Code:

```
"""
Required packages, command: pip install pyttsx3 sounddevice soundfile numpy matplotlib
"""

import pyttsx3
import sounddevice as sd
```

```
import soundfile as sf
import numpy as np
import matplotlib.pyplot as plt

# ---------------- TEXT TO SPEECH ----------------
text = "Welcome to Speech Processing Lab."

engine = pyttsx3.init()
engine.say(text)
engine.runAndWait()

# ---------------- SPEECH RECORDING ----------------
fs = 44100          # Sampling frequency
duration = 5        # Recording time in seconds

print("\nRecording will start now...")
print("Speak clearly for 5 seconds.")

recording = sd.rec(
    int(duration * fs),
    samplerate=fs,
    channels=1,
    dtype='float32'
)

sd.wait()   # Wait until recording is finished

print("Recording finished.")

# Save the recorded speech
filename = "recorded_speech.wav"
sf.write(filename, recording, fs)

print("Speech saved as:", filename)

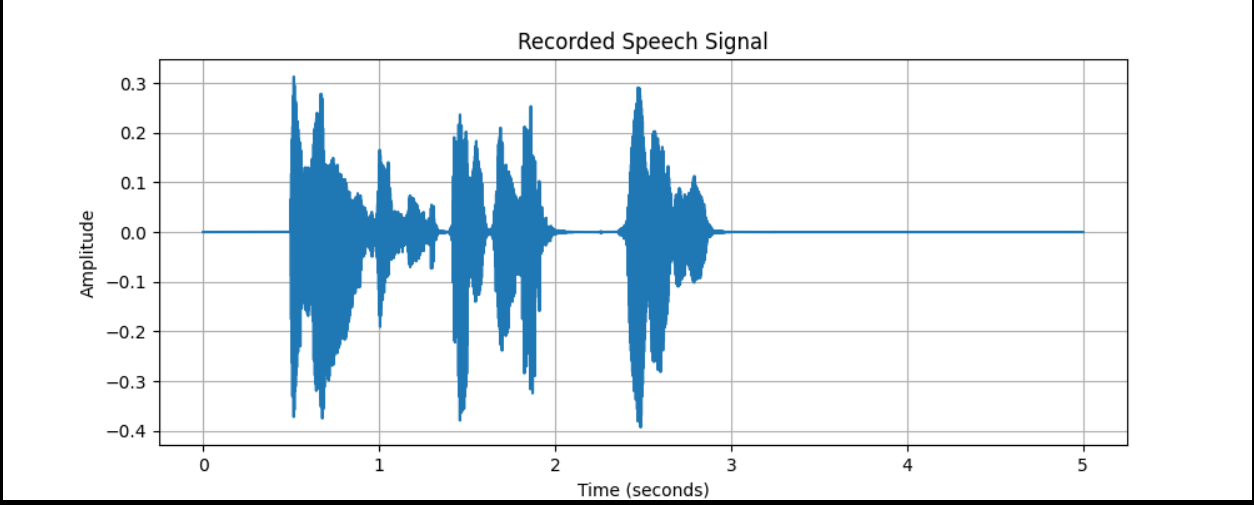
# ---------------- PLAY RECORDED SPEECH ----------------
print("Playing recorded speech...")
sd.play(recording, fs)
sd.wait()
```

```
# ---------------- PLOT SPEECH SIGNAL ----------------
time = np.arange(len(recording)) / fs

plt.figure(figsize=(10, 4))
plt.plot(time, recording)
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.title("Recorded Speech Signal")
plt.grid()
plt.show()
```

Output:

```
Recording will start now...
Speak clearly for 5 seconds.
Recording finished.
Speech saved as: recorded_speech.wav
Playing recorded speech...
```



Result:

The program successfully converted the given text into speech and played it. The speech signal was then recorded through a microphone, saved as an audio file, and played back. The recorded signal was plotted in the time domain.

- Sampling frequency: 44100 Hz
- Recording duration: 5 seconds
- Total number of samples: 220500
- Number of channels: 1 (mono)

Discussion:

In this experiment, text was converted into speech, and the voice was recorded at a sampling frequency of 44100 Hz. The audio was saved, played back, and plotted against time. The graph showed changes in amplitude during speech and silence. Recording quality depended on microphone quality, background noise, and the distance from the microphone.

Conclusion:

The experiment successfully performed text-to-speech conversion, audio recording, saving, playback, and signal plotting. It demonstrated how speech signals can be recorded and analyzed for applications such as speech recognition and speaker identification.

```
"""
    pip install numpy scipy matplotlib PyWavelets
    py -m pip install numpy matplotlib scipy PyWavelets

"""
import numpy as np
import matplotlib.pyplot as plt
import pywt
from scipy.io import wavfile

# 1. Read audio
fs, x = wavfile.read('audio.wav')

if x.ndim > 1:
    x = x.mean(axis=1)

x = x.astype(float)
x = x / np.max(np.abs(x))
t = np.arange(len(x)) / fs

# 2. DWT Decomposition
wavelet = 'db4'
levels = 4

coeffs = pywt.wavedec(x, wavelet, level=levels)

# 3. Reconstruct A4, D4, D3, D2, D1
signals = []

for i in range(len(coeffs)):
    c = [np.zeros_like(a) for a in coeffs]
    c[i] = coeffs[i]
```

```
    signals.append(pywt.waverec(c, wavelet)[:len(x)])

A4 = signals[0]
D4, D3, D2, D1 = signals[1:]

# 4. Find intermediate approximations
A1 = A4 + D4 + D3 + D2
A2 = A4 + D4 + D3
A3 = A4 + D4

approximations = [A1, A2, A3, A4]
details = [D1, D2, D3, D4]

# 5. Plot results
plt.figure(figsize=(11, 12))

plt.subplot(5, 2, 1)
plt.plot(t, x, 'k')
plt.title('Original signal')

for k in range(1, 5):

    plt.subplot(5, 2, 2*k + 1)
    plt.plot(t, approximations[k-1], 'b')
    plt.title(f'Approximation A{k} (0 - {fs/2**(k+1):.0f} Hz)')

    plt.subplot(5, 2, 2*k + 2)
    plt.plot(t, details[k-1], 'r')
    plt.title(f'Detail D{k} ({fs/2**(k+1):.0f} - {fs/2**k:.0f} Hz)')

plt.xlabel('Time (s)')
plt.tight_layout(pad=3)
plt.show()

# 6. Check reconstruction
recon = A4 + D4 + D3 + D2 + D1
print('Max reconstruction error:', np.max(np.abs(x - recon)))
```

```
"""
Q12: Separation of Voiced, Unvoiced and Silence Regions
Method: Short-Time Energy (STE) + Zero Crossing Rate (ZCR)

Install:
pip install numpy scipy matplotlib
```

```
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile

# 1. Read audio
fs, x = wavfile.read('audio.wav')

if x.ndim > 1:
    x = x.mean(axis=1)

x = x.astype(float)
x = x / np.max(np.abs(x))
t = np.arange(len(x)) / fs

# 2. Framing
frame_len = int(0.025 * fs)
hop = int(0.010 * fs)

energy = []
zcr = []

for i in range(0, len(x) - frame_len + 1, hop):
    frame = x[i:i + frame_len]

    energy.append(np.sum(frame ** 2))
    zcr.append(0.5 * np.mean(np.abs(np.diff(np.sign(frame)))))

energy = np.array(energy)
zcr = np.array(zcr)

# 3. Threshold
E_thr = 0.02 * np.max(energy)
Z_thr = 0.15

# 4. Classification
label = []

for i in range(len(energy)):
    if energy[i] < E_thr:
        label.append(0)       # Silence
    elif zcr[i] > Z_thr:
        label.append(1)       # Unvoiced
    else:
        label.append(2)       # Voiced

# 5. Plot
```

```
plt.figure(figsize=(11, 8))

# Original signal
plt.subplot(4, 1, 1)
plt.plot(t, x, 'k')
plt.title('Original Speech Signal')
plt.ylabel('Amplitude')

# Short-time energy
plt.subplot(4, 1, 2)
plt.plot(energy)
plt.axhline(E_thr, color='r', linestyle='--',
            label='Energy Threshold')
plt.title('Short-Time Energy')
plt.legend()

# Zero crossing rate
plt.subplot(4, 1, 3)
plt.plot(zcr)
plt.axhline(Z_thr, color='r', linestyle='--',
            label='ZCR Threshold')
plt.title('Zero Crossing Rate')
plt.legend()

# Voiced, Unvoiced and Silence
plt.subplot(4, 1, 4)

# Legend: show all three labels
plt.plot([], [], color='gray', label='Silence')
plt.plot([], [], color='red', label='Unvoiced')
plt.plot([], [], color='blue', label='Voiced')

for i in range(len(label)):
    start = i * hop
    end = min(start + frame_len, len(x))

    if label[i] == 0:
        plt.plot(t[start:end], x[start:end], color='gray')

    elif label[i] == 1:
        plt.plot(t[start:end], x[start:end], color='red')

    else:
        plt.plot(t[start:end], x[start:end], color='blue')

plt.title('Voiced / Unvoiced / Silence Regions')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
```

```
plt.legend(loc='upper right')

plt.tight_layout()
plt.show()
```